Incremental Maintenance of RDB2RDF Views: A Post-Update, LLM-Compilable Formulation of Changeset Computation

Anonymized Author

Anonymized Institution

Abstract. A strategy for exposing relational data as part of a knowledge graph is to define RDB2RDF views that map relational tuples to RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability, requiring incremental mechanisms to maintain them under updates. A central problem in incremental maintenance is computing deletion changesets, which traditionally requires reconstructing the pre-update database state. This paper challenges this assumption by introducing a formal model of transformation rules that leads to provably correct changesets that operate directly over the post-update state. The paper proceeds by describing how to create an asynchronous maintenance tool for an RDF view, defined by R2RML mappings, leveraging the formal model. The paper first shows how to compile R2RML mappings into transformation rules with the help of an LLM, prompted with the formal model. Then, it introduces an asynchronous maintenance architecture that features AFTER triggers automatically synthesized from the transformation rules, again with the help of an LLM, prompted with the formal model. Finally, the paper describes an experiment that evaluates the complete implementation cycle for an RDF view defined on top of a version of the MusicBrainz database. The paper therefore demonstrates that LLMs, when guided by the proposed formal model, can automatically generate correct database triggers and the maintenance logic for the proposed architecture.

Keywords: LLM SQL Code Generation · Semantic View · Enterprise Knowledge Graph · Formal Specification.

1 Introduction

Enterprise Knowledge Graphs (EKGs) have emerged as a powerful paradigm for integrating heterogeneous enterprise data sources into a unified semantic layer. By linking structured and semi-structured data through an ontology, EKGs enable applications to query and analyze enterprise information in a semantically rich and integrated manner. In many real-world deployments, the underlying enterprise data remains stored in relational databases, while the knowledge graph provides a semantic access layer on top of these sources.

A common strategy for exposing relational data as part of a knowledge graph is the definition of *RDB2RDF views*, which map relational tuples into RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability. However, when the relational source is frequently updated, the materialized RDF view must be continuously synchronized with the underlying database. Recomputing the entire RDF view after each update is typically impractical, motivating the need for *incremental maintenance techniques* that compute only the changes to the view.

A well-established approach for incremental maintenance is the computation of *changesets*, represented as a pair $\langle \Delta^-(u), \Delta^+(u) \rangle$, capturing the triples removed from and added into the RDF view after a relational update u. Prior work has shown that, for entity-preserving RDB2RDF mappings, it is sufficient to identify the *pivot tuples* whose RDF contributions may be affected by the update and recompute their associated RDF state.

The formal framework introduced in [3] provides a rigorous method for computing such changesets based on transformation rules over relational schemas. In particular, in this framework, the deletion component $\Delta^-(u)$ is defined in terms of the pre-update database state. Conceptually, this definition is correct, as it captures precisely the RDF triples that must be removed from the view. However, it introduces a practical limitation when implementing the framework using database triggers, namely, relying on the pre-update database state.

In relational systems, transition tables (e.g., `deleted_R` and `inserted_R`) are only available in *AFTER statement-level triggers*, where the database has already transitioned to the post-update state σ_1. As a consequence, computing $\Delta^-(u)$, according to [3], requires reconstructing portions of the pre-update state σ_0. When the updated relation is large, this reconstruction can be costly, even if the update affects only a small number of tuples.

This paper challenges this assumption by introducing a formal model of changeset computation that operates directly over the post-update state. It shows that, for a transformation rule without multiple occurrences of the same relation, both deletion and insertion components can be derived using only the post-update state, transition tables, and the pre-update view state. For a transformation rule with multiple occurrences of the same relation R, it captures the impact of deletions and insertions on R directly in the evaluation of the transformation rule, still without using the pre-update database state.

The paper proceeds by describing how to create an asynchronous maintenance tool for an RDF view, defined by R2RML mappings, leveraging the formal model. The paper first shows how to compile R2RML mappings into transformation rules with the help of an LLM, prompted with the formal model. Then, it introduces an asynchronous maintenance architecture that selectively decouples SPARQL computation from relational transactions. The architecture avoids executing SPARQL queries inside relational transactions, preventing triplestore latency or temporary unavailability from blocking database updates, while preserving correctness and improving scalability. It then discusses how to compile the required AFTER triggers from the transformation rules, again with the help of an

LLM, prompted with the formal model. In combination, this means that, given an RDF view defined by R2RML mappings, the underlying relational database can be equipped with AFTER triggers that, combined with the other (fixed) components of the architecture, incrementally maintain the RDF view.

Finally, the paper describes an experiment that evaluates the complete implementation cycle for an RDF view defined on top of a version of the MusicBrainz database. The database features a schema with 106 relations, encompassing information about artists, release groups, etc. The RDF view uses the *Music Ontology* vocabulary and is defined by 57 R2RML mappings.

The paper therefore demonstrates that Large Language Models (LLMs), when guided by the proposed formal model, can act as compilers, automatically generating correct database triggers and the maintenance logic for the proposed architecture. The experiments showcase that LLMs are already capable of processing a (complex and lengthy) formal model and correctly output the specified database code, supporting a new paradigm of specification-driven, LLM-based synthesis for incremental RDF view maintenance.

To summarize, the first contribution of this paper is a formal model of the deletion and insertion components of changesets directly over the post-update database state. The second contribution is to describe how to create asynchronous maintenance tools for RDF views, defined by R2RML mappings. The third contribution is to demonstrate that an LLM, guided by appropriate prompts and accessing the formal model, can generate database code that computes and publishes provably correct changesets.

The remainder of this article is organized as follows. Section 2 summarizes related work. Section 3 reviews the basic concepts of RDB2RML views. Section 4 describes a case study. Section 5 introduces the proposed incremental view maintenance formalism. Section 6 discusses an implementation. Section 7 describes the experiments. Section 8 concludes the article. Finally, the appendix contains the proofs of the main results.

2 Related Work

Incremental View Maintenance (IVM). The Incremental View Maintenance problem has been studied in the literature [25] for a very long time.

Early work covered relational views [10], object-oriented views [2], semi-structured views [39], and XML views [15], just to name a few efforts.

Some more recent approaches include the following. As for generic IVM tools, DBSP [9] introduces an expressive language that enables efficient incremental view maintenance for queries written in several languages, such as SQL and Datalog. Enzyme [37] is an IVM engine, developed at Databricks to power Spark Declarative Pipelines, that provides a built-in, end-to-end approach to incremental pipelines, utilizing materialized views as first-class building blocks. OpenIVM [7] is an SQL-to-SQL compiler for Incremental View Maintenance (IVM) that compiles view definitions into SQL to eventually propagate deltas into the table that materializes the view.

As for IVM SPARQL and graph databases, Incremunica [34] integrates multiple incremental operators, allowing it to adapt to different queries and data for optimal performance, and supporting federated querying, dynamically adding and removing sources during query execution, SPARQL Query Language support, multiple IVM techniques, and client-side execution. The counting algorithm, a classic approach to incremental view maintenance for queries on relational data, is adapted in [31] to SPARQL queries on RDF datasets. Graph property views, both virtual and materialized, are considered in [13] to integrate heterogeneous data, using rewriting techniques.

Solutions in specific settings, include the following. The maintainance of self-join-free conjunctive queries over a dynamic database, where tuples can be inserted or deleted, is investigated in [14]. A maintenance approach that combines delta queries, trees of materialized views, and heavy-light data partitioning is introduced in [1], whose update time matches or improves the best update time reported in prior work. The incorporation of knowledge about foreign key into IVM in order to speed up its performance is investifated in [32], which reports experiments in Redshift showing that the proposed technique improved the execution times of the whole refresh process up to 2 times, and up to 2.7 times the process of calculating the necessary changes that will be applied into the materialized view. Finally, a framework for maintaining sketches incrementally under updates is introduced in [17].

The main idea that distinguishes the formal model proposed in this paper from previous work on incremental view maintenance is to explore the entity-preservation property of typical RDB2RDF views, which enables the identification of the specific tuples relevant to a data source update. Only the portion of the view associated with the relevant tuples should be re-materialized.

The proposed formal model revises earlier work [4], which described an incremental maintenance strategy, based on triggers, for RDF views defined on top of relational data. This earlier work was further modified and enhanced to compute changesets for RDB2RDF views [3]. However, these earlier approaches depended on accessing the database state before the update to generate the view changesets. The formal model proposed in this paper remediates this situation and computes the required view changesets from the post-update database state, the current view state, and the update itself.

The definition of the formal model co-evolved with the prompts used in the experiments in the following sense. The experiment started with simple prompts, with the original version of the formal model as an attachment. An analysis of the results indicated that some errors were due to too simple prompts, on the one hand, and to missing or ill-formulated definitions, on the other hand. The analysis also pointed out that the formal definitions could be changed to induce simpler database code. The prompts and the formal model were altered accordingly, and the tests were repeated until satisfactory results were achieved.

Formal Software Engineering, Code Verification, and LLMs. In formal software engineering (SE), LLMs have been employed, for example, to address

formal requirements engineering [12], generate verifiable specifications from informal requirements [8], help with conceptual modeling [19], facilitate software testing [6,35], and build trustworthy AI agents [38].

The application of LLMs to code verification tasks has also received considerable attention [11], for example, including code-invariant generation [21,22] and reasoning about program semantics [16].

Specification-guided LLM Synthesis. Specification-guided LLM synthesis refers to the process of generating executable artifacts from formally defined semantic specifications.

For example, Murphy et al. [24] divided code generation into two parts: one handled by an LLM and the other by formal methods-based program synthesis. VERGE [30] decomposes LLM outputs into atomic claims, autoformalizes them into first-order logic, and verifies their logical consistency using automated theorem proving. SGCR [36] is a framework that grounds LLMs in human-authored specifications to produce trustworthy and relevant feedback.

Specifically with respect to SQL code generation, the text-to-SQL task, that is, the generation of SQL from Natural Language descriptions, has received considerable attention. Recent LLM-based text-to-SQL tools are surveyed in [18,20,28,29]. The Text-to-SQL Handbook[1] lists the best-performing text-to-SQL tools on several benchmarks.

This paper goes in a different direction and shows that LLMs are capable of processing a (complex and lengthy) formal model and correctly output the formally specified database code. Specifically, the paper shows how to compile R2RML mappings into transformation rules with the help of an LLM, prompted with the formal model. It proceeds to demonstrate how to compile the required AFTER triggers from the transformation rules, again with the help of an LLM, prompted with the formal model.

3 RDB2RDF Views

3.1 Basic Concepts and Notation

As usual, a *relation scheme* is denoted as $R[A_1, \ldots, A_n]$. The *relational constraints* considered in this article consist of *mandatory* (or not null) *attributes, keys, primary keys* and *foreign keys*. In particular, $F(R{:}L, S{:}K)$ denotes a foreign key, named F, that *relate* R and S, where L and K are lists of attributes from R and S, respectively, with the same length.

A *relational schema* is a pair $\boldsymbol{S} = (\boldsymbol{R}, \Omega)$, where $\boldsymbol{R}$ is a set of relation schemes and Ω is a set of relational constraints such that: (i) Ω has a unique primary key for each relation scheme in $\boldsymbol{R}$; (ii) Ω has a mandatory attribute constraint for each attribute which is part of a key or primary key; (iii) if Ω has a foreign key of the form $F(R{:}L, S{:}K)$, then Ω also has a constraint indicating that K is

[1] `https://github.com/HKUSTDial/NL2SQL_Handbook`

the primary key of S. The *vocabulary* of $\boldsymbol{S}$ is the set of relation names, attribute names, and foreign key names used in $\boldsymbol{S}$. Given a relation scheme $R[A_1, \ldots, A_n]$ and a *tuple variable* t over R, $t.A_k$ denotes the *projection* of t over A_k. *Selections* over relation schemes are defined as usual.

A *state* σ of a relational schema $\boldsymbol{S}$ assigns to each relation scheme R of $\boldsymbol{S}$ a relation $R(\sigma)$, in the usual way. Let $\boldsymbol{S} = (\boldsymbol{R},\Omega)$ be a relational schema and R and T be relation schemes of $\boldsymbol{S}$.

An *ontology vocabulary*, or simply a *vocabulary*, is a set of *class names, object property names* and *datatype property names*. An *ontology* is a pair $\boldsymbol{O} = (V,\Sigma)$ such that V is a vocabulary and Σ is a finite set of formulae in V, the *constraints* of $\boldsymbol{O}$. The constraints include the definition of the *domain* and *range* of a property, as well as *cardinality constraints*, defined in the usual way.

A *foreign key* is an expression of the form $F(R : K, S : L)$, where:

- F is the *name* of the foreign key;
- R and S are relation schemes;
- K and L are lists of attributes of R and S, respectively;
- $|K| = |L|$.

The relation schemes R and S are said to be *related* by F. The foreign key $F(R : K, S : L)$ induces a binary relationship between tuples of R and S and may be traversed in either direction, from R to S or from S to R. Lastly, a tuple $r \in R(\sigma)$ is related (or connected) to a tuple $s \in S(\sigma)$ with respect to F if and only if the values of the attributes in K of r match the values of the attributes in L of s.

A *relational path* φ is a list of the form $[e_1, \ldots, e_{n-1}]$, where

- e_j is either a foreign key name F_j, indicating that the path traverses F_j from R_j to R_{j+1}, or an expression of the form $(F_j)^{-1}$, indicating that the path traverses the foreign key F_j from R_{j+1} to R_j, in the reverse direction;
- when e_j is traversed in the forward direction, then the second relation in e_j must be equal to the first relation of e_{j+1};
- when e_j is traversed in the reverse direction, then the first relation in e_j must be equal to the first relation of e_{j+1}.

The set of all relation schemes occurring along φ is denoted by $\mathit{Relations}(\varphi)$.

A *prefix* of φ is any sequence $\phi = [e_1, \ldots, e_k]$ with $0 \leq k \leq n-1$.

Given a database state σ, a tuple $r_1 \in R_1(\sigma)$ *is connected to* (or *references*) a tuple $r_n \in R_n(\sigma)$ *by following the path* $\varphi = [e_1, \ldots, e_{n-1}]$ if and only if there exists a sequence of tuples $r_2, \ldots, r_{n-1}$ with $r_i \in R_i(\sigma)$ for $2 \leq i \leq n-1$ such that, for each $i \in \{1, \ldots, n-1\}$, the tuples r_i and r_{i+1} are related with respect to the foreign key F_i in the forward or reverse direction.

A relational path φ induces a *predicate path* $\varphi(x, y)$, which is evaluated in a database state by joining the relations in the path in the directions indicated, using the lists of attributes in the foreign key.

An example of a predicate path with two foreign keys is

$$[(place\_annotation\_fk\_place)^{-1}, \\ place\_annotation\_fk\_annotation](p, an)$$

Note that the first expression $(place\_annotation\_fk\_place)^{-1}$ indicates that the foreign key is traverse in the reverse direction.

3.2 Specification of Entity-Preserving RDB2RDF View

This section presents the formalism for specifying entity-preserving RDB2RDF views. By restricting ourselves to this class of views, we can precisely identify the tuples relevant to a data source update w.r.t. an RDB2RDF view.

The formal definition of an RDB2RDF view is similar to that given in [26,27]. An *RDB2RDF view* is a triple $\mathcal{W} = (V, \boldsymbol{S}, M)$, where:

1. V is the vocabulary of the *target ontology*;
2. $\boldsymbol{S}$ is the *source relational schema*; and
3. M is a set of *mappings* between V and $\boldsymbol{S}$, defined by *transformation rules* (TRs).

A view $\mathcal{W} = (V, \boldsymbol{S}, M)$ satisfies the *entity-preserving property* iff:

- the instances of the classes in V correspond to tuples in selected relations of $\boldsymbol{S}$, which are called the *pivot relations* of $\mathcal{W}$, so that distinct tuples from the same pivot relation generate distinct class instances;
- the values of datatype properties in V of these instances are given by (functions of) attribute values of the corresponding tuples, or of related tuples;
- the object properties in V correspond to relationships between tuples of the pivot relations of $\boldsymbol{S}$.

Intuitively, a view satisfies the entity-preserving property iff it preserves the base entities (objects) of the source database, rather than creating new entities from the existing ones [23].

The problem of generating transformation rules is addressed in [4,5] and is outside of the scope of this work. However, Table 1 summarizes a set of Transformation Rule (TR) patterns that lead to the definition of relational to RDF mappings that guarantees that the RDF views satisfy the entity-preserving property by construction. Table 2 shows the definitions of the concrete predicates used by the TR Patterns in Table 1.

Table 1 defines three types of TR patterns: *Class Transformation Rule* (CTR), *Object Property Transformation Rule* (OTR), and *Datatype Property Transformation Rule* (DTR), further subdivided into *Local DTR* and *Path DTR*.

Intuitively, a CTR ψ maps tuples of a relation R into RDF instances of a class C. It establishes a semantic correspondence between a pivot tuple r and an RDF resource x, such that each pivot tuple is associated with at most one RDF instance, and distinct pivot tuples generate distinct RDF resources. Thus, the mapping preserves the identity of relational entities in the RDF view.

An OTR ψ defines object properties by relating RDF instances generated from pivot tuples to RDF instances reached through relational paths in the source schema. Hence, OTRs capture relationships between entities represented in the relational database.

There are two types of DTRs. A *Local DTR* has no relational path in the body so that the datatype values are obtained directly from attributes values of the pivot tuples. A *Path DTR* has a relational path in the body and the datatype values are obtained from tuples reachable through the relational path from the pivot tuples.

The *pivot relation* of a TR Ψ, denoted $pivot(\Psi)$, is the relation whose tuples serve as the starting point (*pivot tuples*) for the construction of RDF triples. In particular, pivot tuples determine the URI of the subject of the triples generated by Ψ. The body of Ψ uses two auxiliary predicates to handle URIs:

- $hasURI(P_C, PK(r), s)$: this predicate holds iff:
  - P_C is the namespace prefix associated with class C;
  - $PK(r)$ is the value of attributes forming the primary key of tuple r;
  - s is an URI generated by concatenating P_C and the list of values in $PK(r)$.

  The predicate $hasURI$ must guarantee that distinct tuples of R generate distinct URIs.
- $URI\_C(r, s)$: this predicate is introduced by definition, as a short-hand notation, and indicates that s is the URI generated for the tuple $r \in R$:

$$URI\_C(r, s) \equiv hasURI(P_C, PK(r), s)$$

The body of Ψ may also use a *relational path* φ, defined as a list of the form "$[F_1, \ldots F_n]$", where F_j is either the name of a foreign key, indicating that the path traverses F_j from R_j and R_{j+1}, or of the form $(F_j)^{-1}$, indicating that the path traverses F_j from R_{j+1} to R_j, in the reverse direction.

A relational path φ induces a *predicate path* $\varphi(x, y)$, which is evaluated in a database state by joining the relations in the path in the directions indicated. The expression $path(\Psi)$ denotes the relational path of Ψ.

The approach proposed in this article efficiently computes changesets by exploring the entity-preserving property, which allows the precise identification of the tuples in the pivot relations whose corresponding instances may have been affected by an update. The advantages of this approach are: (1) it simplifies the specification of the view, as the formalism is specifically designed to describe the correspondences that define an entity-preserving view; (2) the restrictions on the structure of the queries help ensure the consistency of the specification of the view; (3) the formal expressions that define the queries can be explored to construct the procedures automatically that compute the changesets that maintain the RDB2RDF view; (4) it facilitates the task of providing rigorous proofs for the correctness of the proposed approach.

Table 1. Transformation Rule Patterns

TR	**Transformation Rule Pattern**
CTR	$\psi : C(s) \leftarrow R(r), URI\_C(r, s), \delta(r)$ where - ψ is the name of the transformation rule - R is the name of a relation or a view in the source schema $\boldsymbol{S}$ - r is a tuple variable bound to tuples of R - s is the URI associated with the tuple bound to r - $URI\_C(r, s)$ associates the tuple bound to r to the URI s - $\delta(r)$ is an optional selection over R
OTR	$\psi : P(s, o) \leftarrow R_D(d), B_D[d, s], R_G(g), B_G[g, o], \varphi(d, g)$ where - ψ is the name of the transformation rule - P is an object property in V with domain D and range G - "$R_D(d), B_D[d, s]$" is the RHS of the CTR ψ_D that matches class D with pivot relation R_D of ψ_D - "$R_G(g), B_G[g, o]$" is the RHS of the CTR ψ_G that matches class G with pivot relation R_G of ψ_G - φ is a path from R_D to R_G - $\varphi(d, g)$ holds iff d is connected to g following path φ
Local DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(g.A_1), \ldots, nonNull(g.A_k),$ $RDFLiteral(g.A_1, \text{“}A_1'\text{”}, \text{“}G''\text{”}, v_1), \ldots, RDFLiteral(g.A_k, \text{“}A_k'\text{”}, \text{“}G''\text{”}, v_k),$ $T([v_1, ..., v_k], v)$ where - ψ is the name of the transformation rule - P is a datatype property of V with domain D - "$R_D(r), B_D[r, s]$" is the RHS of the CTR ψ_D that matches class D with pivot relation R_D of ψ_D - $A_1, \ldots, A_k$ are attributes of R_D - T is an optional function that transforms values of attributes $A_1, \ldots, A_k$ of R_D to values of property P
Path DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(g.A_1), \ldots, nonNull(g.A_k),$ $RDFLiteral(g.A_1, \text{“}A_1'\text{”}, \text{“}G''\text{”}, v_1), \ldots, RDFLiteral(g.A_k, \text{“}A_k'\text{”}, \text{“}G''\text{”}, v_k),$ $T([v_1, ..., v_k], v), \varphi(r, g)$ where - ψ is the name of the transformation rule - P is a datatype property of V with domain D - "$R_D(r), B_D[r, s]$" is the RHS of the CTR ψ_D that matches class D with pivot relation R_D of ψ_D - $A_1, \ldots, A_k$ are attributes of R_G - T is an optional function that transforms values of attributes $A_1, \ldots, A_k$ of R_G to values of property P - φ is a path from R_D to R_G - $\varphi(r, g)$ holds iff r is connected to g following path φ

Table 2. Concrete predicates used by the TR Patterns in Table 1

Built-in predicate	**Intuitive definition**
$nonNull(v)$	$nonNull(v)$ holds iff value v is not null
$RDFLiteral(u, A, R, v)$	Given a value u, an attribute A of R, a relation name R, and a literal v, $RDFLiteral(u, A, R, v)$ holds iff v is the literal representation of u, given the type of A in R
$F(r, s)$	Given a tuple r of R and a tuple s of S, $F(r, s)$ holds iff r is related to s by a foreign key of the form $F(R{:}L, S{:}K)$
$hasURI(P_C, A, s)$	Given a tuple r of R, $hasURI(P_C, A, s)$ holds iff s is the URI obtained by concatenating the namespace prefix P_C and the attribute values $a_1, \ldots, a_n$ where A is the list (in Prolog notation) $[a_1, \ldots, a_n]$. To further simplify matters, we admit denoting a list with a single element, "$[a]$", simply as "a".
$URI\_C(r, s)$	is introduced by definition, as a short-hand notation, and indicates that s is the URI generated for the tuple $r \in R$: $URI\_C(r, s) \equiv hasURI(P_C, PK(r), s)$ This predicate conveniently hides the namespace prefix of a class C (which is typically a long string).

3.3 Materialization of an RDB2RDF View

The state of an RDB2RDF view is represented as a set of quadruples (or *quads*) of the form (s, p, o, g). In standard N-Quads, the fourth element g identifies the named graph to which the triple (s, p, o) belongs. In this approach, each transformation rule Ψ is associated with a named graph, and its URI is used as the fourth element. Thus, each quad (s, p, o, g), where g is the URI of a transformation rule Ψ, indicates that the triple (s, p, o) was generated by Ψ.

This representation allows the RDB2RDF view to be materialized as a set of quads partitioned by transformation rules. Intuitively, each rule Ψ induces a named graph containing exactly the triples generated by Ψ, which provides a simple and operational mechanism for provenance tracking and avoids ambiguity in the presence of duplicate triples generated by different rules.

The state of an RDB2RDF View is defined compositionally in three hierarchical levels.

Definition 1 (Hierarchical RDF State Semantics). *Let $W = (V, S, M)$ be an RDB2RDF view and let σ be a relational database state.*

1. ***RDF Contribution produced by a Transformation Rule***
   Let $\Psi \in M$ be a transformation rule whose pivot relation is R^*, with tuple variables $r_1, \ldots, r_n$ associated with relation schemes $R_1, \ldots, R_n$.
   For tuples $p_1 \in R_1(\sigma), \ldots, p_n \in R_n(\sigma)$, the application of Ψ under σ, denoted by

$$\Psi[r_1 \leftarrow p_1, \ldots, r_n \leftarrow p_n](\sigma)$$

is defined as the set of RDF triples produced by evaluating Ψ *with bindings* $r_i = p_i$*, for* $i = 1, \ldots, n$*, over* σ*.*

2. ***RDF Contribution of a Pivot Tuple under a Rule.*** *For a pivot tuple* $t \in R^*(\sigma)$*, the RDF contribution of* t *under* Ψ *and* σ*, denoted by* $RDF\_State[\Psi](t, \sigma)$*, is defined as:*

$$RDF\_State[\Psi](t, \sigma) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow t](\sigma),\ g = URI(\Psi)\}$$

3. ***RDF State of the View W.***
The RDF state of the view W *under* σ*, denoted by* $RDF\_State(W, \sigma)$*, is defined as:*

$$RDF\_State(W, \sigma) = \bigcup_{\Psi \in M} \bigcup_{t \in pivot(\Psi)(\sigma)} RDF\_State[\Psi](t, \sigma)$$

Intuitively, the RDF state of an RDB2RDF view is obtained by aggregating the RDF contributions generated by all transformation rules over all pivot tuples in the database state.

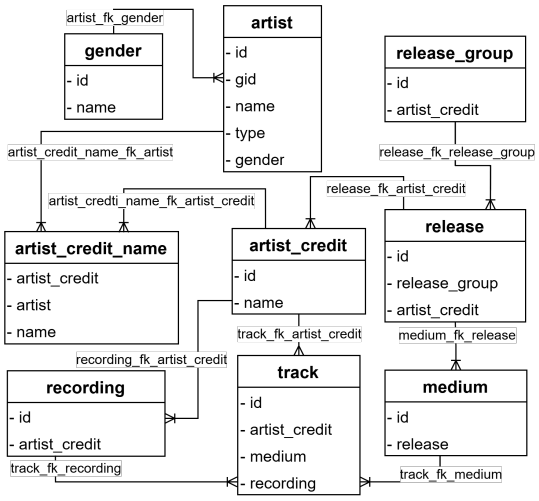
The RDF state can be naturally interpreted as an RDF dataset, where each quad (s, p, o, g) *represents a triple* (s, p, o) *in the named graph identified by* $g = URI(\Psi)$*. Named graphs are used to encode rule-level provenance.*

4 Case Study: *MusicBrainz_RDF*

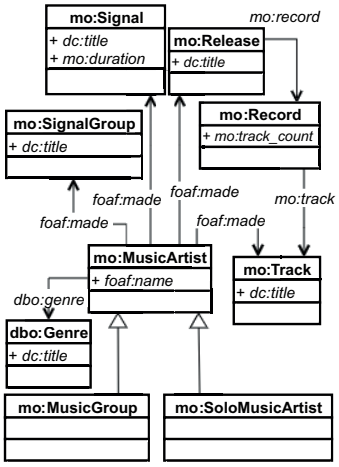
MusicBrainz [33] is an open music encyclopedia that collects music metadata. The ***MusicBrainz*** relational database is built on the PostgreSQL relational database engine and contains all of MusicBrainz music metadata. This data includes information about artists, release groups, releases, recordings, works, and labels, along with the many relationships among them. Figure 1(a) depicts a fragment of the ***MusicBrainz*** relational database schema (for more information about the original scheme, see [33]). Each relation has a distinct primary key, whose name ends with "id", except *gid*, which is an Universally Unique IDentifier (UUID) for use in permanent links and external applications. The relations *Artist, Medium, Release, Recording* and *Track*, in Figure 1(a), represent the main concepts. The relation *ArtistCredit* represents an N:M relationship between *Artist* and *Credit*. The labels of the arcs, such as *fk1*, are the names of the foreign keys.

The case study uses an RDB2RDF view, called ***MusicBrainz_RDF***, which is defined over the relational schema in Figure 1(a). Appendix A provides the Data Definition Language (DDL) for the tables utilized in this case study.

Figure 1(b) depicts a fragment of the ontology used for publishing the ***MusicBrainz_RDF*** view. Appendix B shows the OWL ontology vocabulary used in the case study. It reuses terms from three well-known vocabularies, *FOAF* (Friend of a Friend), *MO* (Music Ontology), and *DC* (Dublin Core).



(a)



(b)

Fig. 1. (a) Fragment of ***MusicBrainz*** Schema and (b) Fragment of ***MusicBrainz_RDF*** View Ontology.

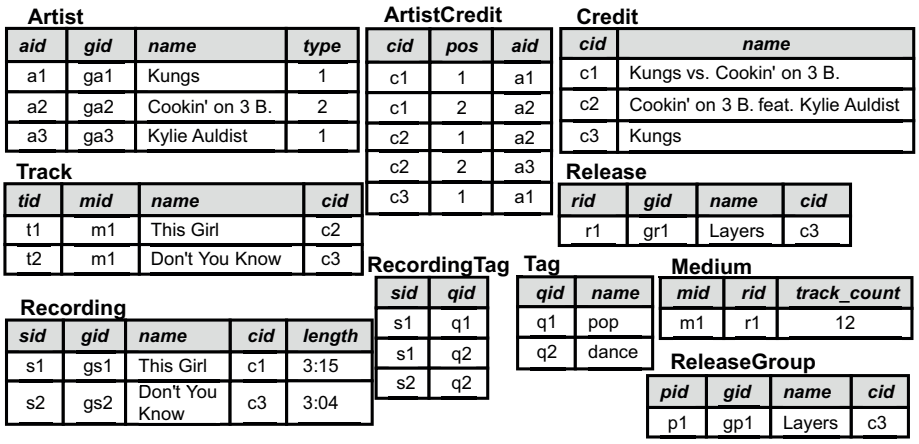


Fig. 2. State Example for Relations in Figure 1(a).

Table 4 shows a set of transformation rules that partially specify the mapping between the relational schema in Figure 1(a) and the ontology in Figure 1(b), obtained with the help of the tool described in [5]. The complete set of transformation rules for the specification of the ***MusicBrainz_RDF*** view is defined in [3] For the examples in the following sections, consider the database state shown in Figure 2.

The transformation rules ψ_1 and ψ_2 in Table 4 are examples of the CTR Pattern. The CTR ψ_1 indicates that, for each tuple r in *Artist*, one should:

1. Compute the URI s such that $hasURI(mbz{:}, r.gid, s)$=true (see Table 2).
2. Produce the triple (s *rdf:type* *mo:MusicArtist*).

The CTR ψ_2 indicates that, for each tuple r in *Artist*, where r.type = 1, one should:

1. Compute the URI s such that $hasURI(mbz{:}, r.gid, s)$=true.
2. Produce the triple (s *rdf:type* *mo:SoloMusicArtist*).

Note that attribute *gid* is a key for relation *Artist*. Therefore, different tuples in *Artist* generate distinct URIs, i.e., distinct instances. Also, note that ψ_1 and ψ_2 use the same predicate *hasURI*. Therefore, if a tuple r is mapped to triples (x *rdf:type* *mo:MusicArtist*) and (y *rdf:type* *mo:SoloMusicArtist*), then x=y.

Considering the database state in Figure 2, CTRs ψ_1 and ψ_2 produce the following triples:

mbz:r.ga1 rdf:type mo:MusicArtist)
(*mbz:r.ga1 rdf:type mo:SoloMusicArtist*)
(*mbz:r.ga2 rdf:type mo:MusicArtist*)
(*mbz:r.ga3 rdf:type mo:MusicArtist*)
(*mbz:r.ga3 rdf:type mo:SoloMusicArtist*)

Table 3. Global URI predicates

Predicate	**Global URI**
URI_artist(a, x)	← *hasURI*("http://musicbrainz.org/artist/", a.gid, x)
URI_area(ar,x)	← *hasURI*("http://musicbrainz.org/area/", ar.gid, x)
URI_label(l,x)	← *hasURI*("http://musicbrainz.org/label/", l.gid, x)
URI_medium(m,x)	← *hasURI*("http://musicbrainz.org/record/", m.id, x)
URI_place(p,x)	← *hasURI*("http://musicbrainz.org/place/", p.gid, x)
URI_recording(r,x)	← *hasURI*("http://musicbrainz.org/recording/", r.gid, x)
URI_release(r,x)	← *hasURI*("http://musicbrainz.org/release/", r.gid, x)
URI_signalGroup(rg,x)	← *hasURI*("http://musicbrainz.org/signal-group/", rg.gid, x)
URI_track(t,x)	← *hasURI*("http://musicbrainz.org/track/", t.id, x)
URI_tag(tg,x)	← *hasURI*("http://musicbrainz.org/tag/", tg.name, x)
URI_work(w,x)	← *hasURI*("http://musicbrainz.org/work/", w.gid, x)
URI_releaseEvent(re,x)	← *hasURI*("http://musicbrainz.org/release/", [re.release_gid, re.country_gid], x) with fragment pattern '#country_gid'
URI_composition(c,x)	← *hasURI*("http://musicbrainz.org/work/", [c.work_gid, "composition"], x)
URI_*_tagging(x,s)	← *hasURI*(entity-prefix, [entity identifier, tag_name], s), matching the R2RML pattern 'entityURI#tag/name'

Table 4. Transformation Rules

TR	Transformation Rules
Ψ_1	$mo{:}MusicArtist(s) \leftarrow artist(a),\ URI\_artist(a,\ s)$
Ψ_2	$mo{:}SoloMusicArtist(s) \leftarrow artist(a),\ URI\_artist(a,\ s),\ (a.type = 1)$
Ψ_3	$mo{:}MusicGroup(s) \leftarrow artist(a),\ URI\_artist(a,\ s),\ (a.type = 2)$
Ψ_4	$mo{:}Record(s) \leftarrow medium(m),\ URI\_medium(m,\ s)$
Ψ_5	$mo{:}Track(s) \leftarrow track(t),\ URI\_track(t,\ s)$
Ψ_6	$mo{:}Release(s) \leftarrow release(r),\ URI\_release(r,\ s)$
Ψ_7	$foaf{:}made(s,\ o) \leftarrow artist(a),\ URI\_artist(a,\ s),\ track(t),\ URI\_track(t,\ o),$ $[artist\_credit\_name\_fk\_artist^{-1},$ $artist\_credit\_name\_fk\_artist\_credit,$ $track\_fk\_artist\_credit^{-1}](a,t)$
Ψ_8	$mo{:}track\ (s,\ g) \leftarrow medium(m),\ URI\_medium(m,\ s),\ track(t),\ URI\_track(t,\ g)$ $[\text{track\_fk\_medium}^{-1}](m,\ t)$
Ψ_9	$mo{:}record(s,\ o) \leftarrow release(r),\ URI\_release(r,\ s),\ medium(m),\ URI\_medium(m,\ o),$ $[medium\_fk\_release^{-1}](r,m)$
Ψ_{10}	$foaf{:}name(s,\ v) \leftarrow artist(a),\ URI\_artist(a,\ s),\ nonNull(a.name),$ $RDFLiteral(a.name,\ \text{"name"},\ \text{"artist"},\ v)$
Ψ_{11}	$mo{:}track\_count(s,\ v) \leftarrow medium(m), URI\_medium(m,\ s),\ nonNull(m.track\_count),$ $RDFLiteral(m.track\_count,\ \text{"track\_count"},\ \text{"medium"},\ v)$
Ψ_{12}	$dc{:}title(s,\ v) \leftarrow track(t),\ URI\_track(t,\ s),\ nonNull(t.name),$ $RDFLiteral(t.name,\ \text{"name"},\ \text{"track"},\ v)$
Ψ_{13}	$dc{:}title(s,\ v) \leftarrow release(r),\ URI\_release(r,\ s),\ nonNull(r.name),$ $RDFLiteral(r.name,\ \text{"name"},\ \text{"release"},\ v)$
Ψ_{14}	$mo{:}Signal(s) \leftarrow recording(r),\ URI\_recording(r,\ s)$
Ψ_{15}	$mo{:}SignalGroup(s) \leftarrow release\_group(rg),\ URI\_signalGroup(rg,\ s)$
Ψ_{16}	$dbo{:}Genre(s) \leftarrow tag(tg),\ URI\_tag(tg,\ s)$
Ψ_{17}	$dc{:}title(s,\ v) \leftarrow recording(r),\ URI\_recording(r,\ s),\ nonNull(r.name),$ $RDFLiteral(r.name,\ \text{"name"},\ \text{"recording"},\ v)$
Ψ_{18}	$mo{:}duration(s,\ v) \leftarrow track(t),\ URI\_track(t,\ s),\ nonNull(t.length),$ $RDFLiteral(t.length,\ \text{"length"},\ \text{"track"},\ v)$
Ψ_{19}	$dc{:}title(s,\ v) \leftarrow release\_group(rg),\ URI\_signalGroup(rg,\ s),\ nonNull(rg.name),$ $RDFLiteral(rg.name,\ \text{"name"},\ \text{"release\_group"},\ v)$
Ψ_{20}	$dc{:}title(s,\ v) \leftarrow tag(tg),\ URI\_tag(tg,\ s),\ nonNull(tg.name),$ $RDFLiteral(tg.name,\ \text{"name"},\ \text{"tag"},\ v)$
Ψ_{21}	$foaf{:}made(s,\ o) \leftarrow artist(a),\ URI\_artist(a,\ s),\ release(r),\ URI\_release(r,\ o)$ $[artist\_credit\_name\_fk\_artist^{-}1,$ $artist\_credit\_name\_fk\_artist\_credit,$ $release\_fk\_artist\_credit^{-}1](a,r)$
Ψ_{22}	$foaf{:}made(s,\ o) \leftarrow artist(a),\ URI\_artist(a,\ s),\ release\_group(rg),\ URI\_signalGroup(rg,\ o)$ $[artist\_credit\_name\_fk\_artist^{-}1,$ $artist\_credit\_name\_fk\_artist\_credit,$ $release\_group\_fk\_artist\_credit^{-}1](a,\ rg)$
Ψ_{23}	$foaf{:}made(s,\ o) \leftarrow artist(a),\ URI\_artist(a,\ s),\ recording(r),\ URI\_recording(r,\ o)$ $[artist\_credit\_name\_fk\_artist^{-}1,$ $artist\_credit\_name\_fk\_artist\_credit,$ $recording\_fk\_artist\_credit^{-}1](a,\ r)$
Ψ_{24}	$dbo{:}genre(s,\ v) \leftarrow artist(a),\ URI\_artist(a,\ s),\ gender(g),$ $nonNull(g.name),\ lower(g.name,\text{u}),\ RDFLiteral(\text{u,"name","gender"},v),$ $[artist\_fk\_gender](a,g)$

The transformation rule ψ_{10}, in Table 4, is an example of the DTR Pattern. The predicates *nonNull*(v) and *RDFLiteral*(u, A, R, v) are defined in Table 2. Rule ψ_{10} matches the value of attribute *name* of relation *Artist* with the value of datatype property *foaf:name*, whose domain is *mo:MusicArtist*. It indicates that, for each tuple r of R, one should:

1. Compute the URI s for the instance of *mo:MusicArtist* that r represents, using the CTR ψ_1. Therefore, s and r are semantically equivalent.
2. For each value v, where v is the literal representation of *r.name* and *r.name* is not NULL, produce triple (s *foaf:name* v).

Considering the database state in Figure 2, ψ_{10} produces the following triples:

(*mbz:ga1 foaf:name "Kungs"*)
(*mbz:ga2 foaf:name "Cookin's on 3 B."*)
(*mbz:ga3 foaf:name "Kylie Auldist"*)

The transformation rules ψ_7 and ψ_8, in Table 4, are examples of the OTR Pattern. The predicate $F(r, s)$, where F is a foreign key of the form $F(R{:}L, S{:}K)$, is defined in Table 2. For example, rule ψ_7 matches a relationship between a tuple r of *Artist* and a tuple r_3 in *Track* to instances of the object property *foaf:made*, whose domain is *mo:MusicArtist* and range is *mo:Track*. It indicates that, for each tuple r of R, one should:

1. Compute the URI s for the instance of *mo:MusicArtist* that r represents, using the CTR ψ_1. Therefore, s and r are semantically equivalent.
2. For each tuple r_3 of *Track* such that r is related to r_3 through path

$$[(artist\_credit\_name\_fk\_artist)^{-1},\ artist\_credit\_name\_fk\_artist\_credit,\ (track\_fk\_artist\_credit)^{-1}](r, t)$$

   compute the URI g for the instance of *mo:Track* that r_3 represents (using CTR ψ_5). Therefore, r_3 and g are semantically equivalent.
3. Produce triple (s *foaf:made* g)

Considering the database state in Figure 2, ψ_7 produces the following triples:

(*mbz:ga1 foaf:made mbz:t2*)
(*mbz:ga2 foaf:made mbz:t1*)
(*mbz:ga3 foaf:made mbz:t1*)

Example 1. Consider the transformation rules for the ***MusicBrainz_RDF*** view defined in Table 4. Also, consider the relation *Artist* in Figure 1(a), which is the pivot relation of the Transformation Rules (TRs) Ψ_1, Ψ_2, Ψ_3, Ψ_7, Ψ_{10}, and Ψ_{24}. Thus, the RDF state of a tuple p in relation *Artist* is computed by applying the TRs $\Psi_1[r/p]$, $\Psi_2[r/p]$, $\Psi_3[r/p]$, $\Psi_7[r/p]$, $\Psi_{10}[r/p]$, and $\Psi_{24}[r/p]$. Considering

the database state in Figure 2, the RDF state of tuple *a1* in *Artist* contains the following quads:

$(mbz{:}ga1 \;\; rdf{:}type \;\; mo{:}MusicArtist \;\; \Psi_1)$
$(mbz{:}ga1 \;\; rdf{:}type \;\; mo{:}SoloMusicArtist \;\; \Psi_2)$
$(mbz{:}ga1 \;\; foaf{:}made \;\; mbz{:}t2 \;\; \Psi_7)$
$(mbz{:}ga1 \;\; foaf{:}name \;\; \text{“}Kungs\text{”} \;\; \Psi_{10})$
$(mbz{:}ga1 \;\; dbo{:}genre \;\; mbz{:}q1 \;\; \Psi_{24})$
$(mbz{:}ga1 \;\; dbo{:}genre \;\; mbz{:}q2 \;\; \Psi_{24})$
$(mbz{:}ga1 \;\; dbo{:}genre \;\; mbz{:}q2 \;\; \Psi_{24})$

We use Ψ interchangeably to denote both a transformation rule and its associated URI.

Note that, $\Psi_3[r/a1]$ produces no triple, while $\Psi_{24}[r/a1]$ produces duplicated triples. The triple (mbz:ga1 *dbo:genre* mbz:gq2) is generated twice by assigning different tuple variables in Ψ_{24}. Duplicate triples arise when multiple join bindings of the same transformation rule produce identical RDF triples. This typically occurs when different relational paths or join combinations connect the same pivot tuple to equivalent target tuples.

5 Post-Update Incremental Maintenance of RDB2RDF Views

5.1 The Incremental Maintenance Problem

The diagram in Figure 3 describes the problem of computing a correct changeset for a materialized RDB2RDF view $\mathcal{W}$, when an update u occurs in the source relational database $\boldsymbol{S}$. In the diagram of Figure 3, assume that:

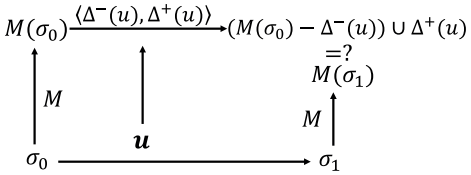


Fig. 3. The problem of computing the correct changeset for RDB2RDF View.

1. M is the set of mappings that materialize view $\mathcal{W}$;
2. σ_0 and σ_1 are the states of $\boldsymbol{S}$ respectively before and after the update u; and
3. $M(\sigma_0)$=$RDF\_State(\mathcal{W}, \sigma_0)$ and $M(\sigma_1)$= $RDF\_State(\mathcal{W}, \sigma_1)$ are the materializations of $\mathcal{W}$ respectively at σ_0 and σ_1.

Let u be an update and σ_0 and σ_1 be the database states before and after u, respectively. A *correct changeset* for $\mathcal{W}$ w.r.t. u, σ_0 and σ_1 is a pair $\langle \Delta^-(u), \Delta^+(u)\rangle$, where $\Delta^-(u)$ is a set of triples removed from $\mathcal{W}$ and $\Delta^+(u)$ is a set of triples added to $\mathcal{W}$, that satisfies the following restriction (see Figure 3):

$$M(\sigma_1) = (M(\sigma_0) \setminus \Delta^-(u)) \cup \Delta^+(u) \tag{1}$$

Putting this in words, the changeset $\langle \Delta^-(u), \Delta^+(u)\rangle$ is correctly computed iff the new view state $(M(\sigma_0) \setminus \Delta^-(u)) \cup \Delta^+(u)$, computed with the help of the changeset, and the new view state $M(\sigma_1)$, obtained by rematerializing the view using the view mappings are identical. In this section, we prove that the incremental formulation of changeset computation preserves the semantic effect of the original framework. Specifically, we show that applying the incremental changeset to the pre-update RDF state yields exactly the RDF state of the view after the update.

Database triggers may be defined to compute and publish the correct changeset for the RDB2RDF view, ensuring that the RDF materialization remains synchronized with the underlying relational database [3].

The first step of the strategy is to identify the *relations in the source database that are relevant to the RDB2RDF view*, that is, those relations whose updates may affect the RDF state of the view.

For each update operation u performed on a relevant relation R, a single statement-level `AFTER` trigger is defined. The `AFTER` trigger is fired immediately after the execution of the update and is responsible for computing $\Delta^-(u)$.

Although $\Delta^-(u)$ is formally defined over the pre-update database state σ_0, its computation is performed inside an `AFTER` trigger, where the database is already in the post-update state σ_1. Thus, even though the trigger executes after the update, the combination of the deleted and inserted tuple sets with the current database state and old view state allows to correctly compute the changesets. Consequently, $\Delta^-(u)$ is computed precisely as defined over the pre-update database state.

5.2 A Post-Update Formulation of Changeset Computation

Let $u = (D, I)$ be an update over a relation R, let σ_1 be the database state after applying u, and let $\mathcal{W}$= (V, $\boldsymbol{S}$, M) be an RDB2RDF view. In most definitions, an explicit mention to σ_1 will be omitted to simplify the notation slightly.

First, define that a transformation rule $\Psi \in M$ is *relevant* to R if and only if $pivot(\Psi) = R$, or $R \in Relations(path(\Psi))$. The set of transformation rules relevant to R is denoted $Relev(R)$.

If $pivot(\Psi) = R$, then Ψ is said to be *pivot-relevant* to R. In this case, the tuples in D and I are themselves pivot tuples of Ψ, and their RDF contributions are directly affected by the update.

If $R \in Relations(path(\Psi))$, then Ψ is said to be *relation-relevant* to R. In this case, the update affects the evaluation of Ψ indirectly: tuples in the pivot relation $R^* = pivot(\Psi)$ whose derivations depend on tuples in D or I may have their RDF contributions modified.

Importantly, the two conditions above are not mutually exclusive: a transformation rule Ψ may be both pivot-relevant and relation-relevant to R.

The changesets $\Delta^-(u)$ and $\Delta^+(u)$ are computed by aggregating the deletion and insertion contributions of all transformation rules that are relevant to the update. Intuitively, each relevant rule $\Psi \in M$ contributes with a pair

$$< \Delta^-[\Psi](u), \Delta^+[\Psi](u) >$$

where

- $\Delta^-[\Psi](u)$: the incremental deletion contribution is the set of RDF quads that become invalid due to the deletion of tuples in D
- $\Delta^+[\Psi](u)$: the incremental insertion contribution is the set of RDF quads that become valid due to the insertion of tuples in I

The overall changeset is then defined as:

$$\Delta^-(u) = \bigcup_{\Psi \in Relev(R)} \Delta^-[\Psi](u)$$

$$\Delta^+(u) = \bigcup_{\Psi \in Relev(R)} \Delta^+[\Psi](u)$$

In the following subsections, we detail how to compute the incremental deletion and insertion contributions $\Delta^-[\Psi](u)$ and $\Delta^+[\Psi](u)$.

5.2.1 Computing the Incremental Deletion Contribution of Relevant Rules

For a transformation rule $\Psi \in M$ that is relevant to an update u over relation R, the incremental deletion contribution can be decomposed into two components:

$$\Delta^-[\Psi](u) = \Delta^-_{pivot}[\Psi](u) \cup \Delta^-_{rel}[\Psi](u)$$

where

- $\Delta^-_{pivot}[\Psi](u)$: denotes the deletion contribution of Ψ arising from its pivot-relevant effect.
- $\Delta^-_{rel}[\Psi](u)$: denotes the deletion contribution of Ψ arising from its relation-relevant effect.

In the following, we first describe how to compute $\Delta^-_{pivot}[\Psi](u)$ and then $\Delta^-_{rel}[\Psi](u)$.

5.2.1.1 Incremental Deletion Contribution of Pivot-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0. Consider that R is the pivot relation of ψ (that is, R is the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is pivot-relevant to R.

Let $d \in D$ be a deleted pivot tuple. In this case, each RDF triple (s, p, o) generated from d by Ψ in σ_0 becomes invalid, and the quad (s, p, o, g) must be

removed from the view, where $g = URI(\Psi)$ encodes the provenance of triples generated by rule Ψ.

In detail, *the incremental contribution of Ψ with respect to a deleted tuple $d \in D$ from relation R*, when Ψ is pivot-relevant to R, is defined as:

$$\Delta^{-}_{pivot}[\Psi](d) = \{(s,p,o,g) \mid (s,p,o,g) \in W_0,\ s = URI(d),\ g = URI(\Psi)\}$$

where W_0 is the view state before the update, $URI(d)$ denotes the RDF subject associated with the pivot tuple d, and $g = URI(\Psi)$.

Then, *the overall incremental contribution of a rule Ψ with respect to all deleted tuples in an update $u = (D, I)$* on R, when Ψ is pivot-relevant to R, is defined by aggregating the contributions of all deleted pivot tuples:

$$\Delta^{-}_{pivot}[\Psi](u) = \bigcup_{d \in D} \Delta^{-}_{pivot}[\Psi](d)$$

Intuitively, since the pivot tuple itself is deleted, all RDF triples previously generated from that tuple become invalid. Under the assumption of entity-preserving mappings, each pivot tuple generates a disjoint set of triples, as distinct pivot tuples are mapped to distinct RDF subjects. Therefore, the deletion contribution can be computed independently for each $d \in D$, and the overall result is simply the union of these per-tuple contributions. (See also Lemma 1 in Section 5.4.)

5.2.1.2 Incremental Deletion Contribution of Relation-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0. Consider an occurrence of R which is not the pivot relation of ψ (that is, the occurrence of R is not the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is relation-relevant to R. Let R^* be the pivot relation of Ψ.

Let $d \in D$ be a deleted tuple. Then, the update indirectly affects the evaluation of Ψ. Therefore, the computation of the incremental deletion contribution proceeds in two steps: (i) identifying the set of pivot tuples indirectly affected by the deleted tuples D; and (ii) computing the deletion contribution associated with those affected pivot tuples.

Let $p^* \in R^*(\sigma_0)$. The pivot tuple p^* is said to be *indirectly affected by $u = (D, I)$* iff its RDF contribution depends on at least one tuple in D through the evaluation of Ψ in σ_0, formally defined as:

$$\begin{aligned} A^{-}[\Psi](u) = \{p^* \in R^*(\sigma_0) \mid\ & \exists d \in D,\ \exists r_k \in VarR(\Psi), \\ & \exists (s,p,o) \in \Psi[r^* \leftarrow p^*,\ r_k \leftarrow d](\sigma_0)\} \end{aligned}$$

where $VarR(\Psi)$ is the set of tuple variables in Ψ associated with relation R.

Intuitively, $A[\Psi](u)$ contains all pivot tuples $p^* \in R^*(\sigma_0)$ such that at least one deleted tuple $d \in D$, when bound to some variable r_k of rule Ψ associated with relation R, contributes to the generation of at least one RDF triple by Ψ in the pre-update database state σ_0.

This definition refers to the pre-update database state σ_0, but it can be rephrased in terms of the post-update database state σ_1 as follows.

Let $\sigma[R \leftarrow S]$ denote the database state σ' which is equal to σ, except that $R(\sigma') = S$. Then, $A^-[\Psi](u)$, with $u = (D, I)$, can rephrased as:

$$A^-[\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists d \in D,\ \exists r_k \in VarR(\Psi),\\ \exists (s,p,o) \in \Psi[r^* \leftarrow p^*,\ r_k \leftarrow d](\sigma_1[R \leftarrow ((R \cup D) \setminus I]))\}$$

Note that $\Psi[r^* \leftarrow p^*,\ r_k \leftarrow d](\sigma_1[R \leftarrow ((R \cup D) \setminus I)])$ simply says that

- the tuple variable r^* is instantiated with the pivot tuple p^*.
- the tuple variable r_k, associated with some occurrence of R in $path(\Psi)$, is instantiated with some deleted tuple d.
- all other tuple variables associated with other occurrences of R in $path(\Psi)$ are instantiated with tuples from $((R(\sigma_1) \cup D) \setminus I)$, that is, with tuples from $R(\sigma_0)$, since $((R(\sigma_1) \cup D) \setminus I) = R(\sigma_0)$.

In fact, one can prove that the two definitions are equivalent, using two simple facts:

- $R^*(\sigma_0) = R^*(\sigma_1)$, since by assumption $R^* \neq R$ and u is an update on R.
- $\sigma_1[R \leftarrow ((R \cup D) \setminus I)] = \sigma_0$, since $u = (D, I)$ is an update on R and, hence, it follows that $(R(\sigma_1) \cup D \setminus I) = R(\sigma_0)$, and $T(\sigma_1) = T(\sigma_0)$, for any other relation scheme $T \neq R$.

There is also a special case worth considering when R occurs only once in $path(\Psi)$, in which case $A^-[\Psi](u)$ can be equivalently rewritten as:

$$A^-[\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists d \in D,\\ \exists (s,p,o) \in \Psi[r^* \leftarrow p^*,\ r_R \leftarrow d](\sigma_1)\}$$

where r_R is the tuple variable associated with the only occurrence of R in $path(\Psi)$.

This follows because there are no other tuple variables associated with other occurrences of R in $path(\Psi)$ to be instantiated. Hence,

$$\Psi[r^* \leftarrow p^*,\ r_k \leftarrow d](\sigma_1) = \Psi[r^* \leftarrow p^*,\ r_k \leftarrow d](\sigma_0)$$

Now, let $p^* \in A^-[\Psi](u)$ and W_0 be the view state before the update. Consider the sets:

$$S_1[\Psi](p^*) = \{(s,p,o,g) \mid (s,p,o,g) \in W_0,\ s = URI(p^*),\ g = URI(\Psi)\}$$

$$S_2[\Psi](p^*) = \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1),\ g = URI(\Psi)\}$$

where r^* is the tuple variable associated with the pivot relation R^*.

The set $S_1[\Psi](p^*)$ contains all quads generated by Ψ from p^* in σ_0 (since $(s,p,o) \in W_0$, $s = URI(p^*)$ and $g = URI(\Psi)$), whereas $S_2[\Psi](p^*)$ contains all quads generated by Ψ from p^* in the post-update database state σ_1. Note that $S_1[\Psi](p^*)$ may include a quad q that is not affected by the update, which is

precisely one of the quads in $S_2[\Psi](p^*)$. Therefore, q is preserved by the set difference operation.

The *incremental deletion contribution of Ψ with respect to $p^* \in A^-[\Psi](u)$*, with $u = (D, I)$, when Ψ is relation-relevant to R, is defined as:

$$\Delta^-_{rel}[\Psi](p^*) = (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

The overall incremental deletion contribution of a rule $\Psi \in \mathcal{M}$ for $u = (D, I)$, when Ψ is relation-relevant to R, is defined by aggregating the contributions of all affected pivot tuples:

$$\Delta^-_{rel}[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} \Delta^-_{rel}[\Psi](p^*)$$

5.2.2 Computing the Incremental Insertion Contribution of Relevant Rules

For a transformation rule $\Psi \in M$ that is relevant to an update u over relation R, the incremental insertion contribution can be decomposed into two components:

$$\Delta^+[\Psi](u) = \Delta^+_{pivot}[\Psi](u) \cup \Delta^+_{rel}[\Psi](u)$$

where

- $\Delta^+_{pivot}[\Psi](u)$: denotes the insertion contribution of Ψ arising from its pivot-relevant effect.
- $\Delta^+_{rel}[\Psi](u)$: denotes the insertion contribution of Ψ arising from its relation-relevant effect

In the following, we first describe how to compute $\Delta^+_{pivot}[\Psi](u)$ and then $\Delta^+_{rel}[\Psi](u)$.

5.2.2.1 Incremental Insertion Contribution of Pivot-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0. Consider that R is the pivot relation of ψ (that is, R is the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is pivot-relevant to R.

The *incremental contribution of Ψ with respect to an inserted tuple $i \in I$ into relation R*, when Ψ is pivot-relevant to R, is defined as:

$$\Delta^+_{pivot}[\Psi](i) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow i](\sigma_1),\ g = URI(\Psi)\}$$

where $\Psi[r \leftarrow i](\sigma_1)$ denotes the set of triples generated by applying rule Ψ to the pivot tuple i, evaluated over the post-update database state σ_1.

Then, *the overall incremental contribution of a rule Ψ with respect to all inserted tuples in an update $u = (D, I)$* on R, when Ψ is pivot-relevant to R, is defined by aggregating the contributions of all inserted pivot tuples:

$$\Delta^+_{pivot}[\Psi](u) = \bigcup_{i \in I} \Delta^+_{pivot}[\Psi](i)$$

5.2.2.2 Incremental Insertion Contribution of Relation-Relevant Rules

Let Ψ be a transformation rule and $u = (D, I)$ be an update on R in state σ_0. Consider an occurrence of R which is not the pivot relation of ψ (that is, the occurrence of R is not the first occurrence of a relation scheme in $path(\Psi)$), in which case ψ is relation-relevant to R. Let R^* be the pivot relation of Ψ.

In this case, the update affects the evaluation of Ψ indirectly. Therefore, the computation of the incremental insertion contribution proceeds in two steps: (i) identifying the set of pivot tuples indirectly affected by the inserted tuples I; and (ii) computing the insertion contribution associated with those affected pivot tuples.

A pivot tuple is said to be *indirectly affected* if it is not itself inserted, but its RDF contribution depends on one or more tuples in I through the evaluation of Ψ, formally defined as:

$$A^+\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists i \in I,\ \exists r_k \in VarR(\Psi), \\ \exists (s,p,o) \in \Psi[r^* \leftarrow p^*,\ r_k \leftarrow i](\sigma_1)\}$$

where:

- $R^* = \text{pivot}(\Psi)$
- r^* is the tuple variable associated with R^*
- $VarR(\Psi)$ denotes the set of tuple variables in Ψ associated with relation R

There is also a special case worth considering when R occurs only once in $path(\Psi)$, in which case $A^+[\Psi](u)$ can be equivalently rewritten as:

$$A^+[\Psi](u) = \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \\ \exists (s,p,o) \in \Psi[r^* \leftarrow p^*,\ r_R \leftarrow i](\sigma_1)\}$$

where r_R is the tuple variable associated with the only occurrence of R in $path(\Psi)$.

Then, the *incremental insertion contribution of Ψ with respect to $p^* \in A^+[\Psi](u)$* is defined as:

$$\Delta^+_{rel}[\Psi](p^*) = \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1),\ g = URI(\Psi)\}$$

The set $\Delta^+_{rel}[\Psi](p^*)$ contains all quads generated by Ψ from p^* in the post-update database state σ_1. Note that duplicate quads may arise; however, this is harmless under the set semantics of RDF. As a result, only the quads that become valid due to the insertion of tuples in I are inserted.

Finally, *the overall incremental insertion contribution of a rule $\Psi \in \mathcal{M}$ for u*, when Ψ is relation-relevant to R, is defined by aggregating the contributions of all affected pivot tuples:

$$\Delta^+_{rel}[\Psi](u) = \bigcup_{p^* \in A^+[\Psi](u)} \Delta^+_{rel}[\Psi](p^*)$$

5.3 An Abstract Algorithm Synthesized from the Formal Definitions

This section presents an abstract algorithm (Algorithm 1) to compute the insertion and deletion changesets $\Delta^{-}(u)$ and $\Delta^{+}(u)$. The algorithm basically rewrites the formal definitions in pseudo-code, following a compositional structure, and ensuring that all affected RDF quads are computed using the post-update state σ_1 and the pre-update RDF view state W_0.

Importantly, the algorithm was automatically synthesized from the formal definitions using an LLM. This illustrates a key property of the proposed framework, namely *LLM compilability*: the ability of a formal specification to be directly translated into an operational algorithm by an LLM. This suggests a new methodological direction in data management, where formal theories are designed not only for correctness, but also for machine-driven compilation into executable systems.

The algorithm presented in this section is intentionally abstract and should not be interpreted as concrete implementations for generating database triggers. Rather, it provides a procedural interpretation of the formal definitions, serving as a bridge between the declarative specification of the framework and its operational realization. In this sense, the algorithm can be understood as a form of *meta-level code*.

In our approach, LLMs play two complementary roles. First, they are used to synthesize abstract algorithms from the formal definitions. This process acts as a form of validation, ensuring that the definitions are sufficiently precise, complete, and internally consistent to be translated into an operational procedure without requiring additional implicit assumptions.

Second, given a concrete specification of the mappings and the target environment, an LLM can be used to generate executable trigger implementations directly from the same formal definitions. This second role is discussed in the following section.

Algorithm 1: COMPUTE_DELTA_R(W, u, R, σ_1, W_0)

Input: RDB2RDF view $W = (V, S, M)$; update $u = (D, I)$ over relation R; post-update database state σ_1; pre-update RDF state W_0.
Output: Changeset $\langle \Delta^-(u), \Delta^+(u) \rangle$.

$\Delta^-(u), \Delta^+(u) \leftarrow \emptyset$;
$Relev(R) \leftarrow \{\Psi \in M \mid pivot(\Psi) = R \ \vee \ R \in Relations(path(\Psi))\}$;
foreach $\Psi \in Relev(R)$ **do**
  $\Delta^-_{pivot}[\Psi](u), \Delta^-_{rel}[\Psi](u) \leftarrow \emptyset$;
  $\Delta^+_{pivot}[\Psi](u), \Delta^+_{rel}[\Psi](u) \leftarrow \emptyset$;
  `// 1. Pivot-relevant contribution`
  if $pivot(\Psi) = R$ **then**
    foreach $d \in D$ **do**
      $\Delta^-_{pivot}[\Psi](d) \leftarrow \{(s,p,o,g) \in W_0 \mid s = URI(d) \wedge g = URI(\Psi)\}$;
      $\Delta^-_{pivot}[\Psi](u) \leftarrow \Delta^-_{pivot}[\Psi](u) \cup \Delta^-_{pivot}[\Psi](d)$;
    end
    foreach $i \in I$ **do**
      $\Delta^+_{pivot}[\Psi](i) \leftarrow \{(s,p,o,g) \mid (s,p,o) \in \Psi[r \leftarrow i](\sigma_1) \wedge g = URI(\Psi)\}$;
      $\Delta^+_{pivot}[\Psi](u) \leftarrow \Delta^+_{pivot}[\Psi](u) \cup \Delta^+_{pivot}[\Psi](i)$;
    end
  end
  `// 2. Relation-relevant contribution`
  if $R \in Relations(path(\Psi))$ **then**
    $R^* \leftarrow pivot(\Psi)$;
    `// 2.1 Affected pivot tuples for deletion`
    if *R occurs only once in $path(\Psi)$* **then**
      $A^-_\Psi \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \ \exists (s,p,o) \in \Psi[r^* \leftarrow p^*, r_R \leftarrow d](\sigma_1)\}$;
    end
    else
      $\sigma^0_R \leftarrow (R(\sigma_1) \cup D) \setminus I$;
      $A^-_\Psi \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists r_k \in Var_R(\Psi), \ \exists (s,p,o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1[R \leftarrow \sigma^0_R])\}$;
    end
    foreach $p^* \in A^-_\Psi$ **do**
      $S_1[\Psi](p^*) \leftarrow \{(s,p,o,g) \in W_0 \mid s = URI(p^*) \wedge g = URI(\Psi)\}$;
      $S_2[\Psi](p^*) \leftarrow \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$;
      $\Delta^-_{rel}[\Psi](p^*) \leftarrow S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)$;
      $\Delta^-_{rel}[\Psi](u) \leftarrow \Delta^-_{rel}[\Psi](u) \cup \Delta^-_{rel}[\Psi](p^*)$;
    end
    `// 2.2 Affected pivot tuples for insertion`
    $A^+_\Psi \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \exists r_k \in Var_R(\Psi), \ \exists (s,p,o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow i](\sigma_1)\}$;
    foreach $p^* \in A^+_\Psi$ **do**
      $\Delta^+_{rel}[\Psi](p^*) \leftarrow \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$;
      $\Delta^+_{rel}[\Psi](u) \leftarrow \Delta^+_{rel}[\Psi](u) \cup \Delta^+_{rel}[\Psi](p^*)$;
    end
  end
  $\Delta^-[\Psi](u) \leftarrow \Delta^-_{pivot}[\Psi](u) \cup \Delta^-_{rel}[\Psi](u)$
  $\Delta^+[\Psi](u) \leftarrow \Delta^+_{pivot}[\Psi](u) \cup \Delta^+_{rel}[\Psi](u)$
  $\Delta^-(u) \leftarrow \Delta^-(u) \cup \Delta^-[\Psi](u)$;
  $\Delta^+(u) \leftarrow \Delta^+(u) \cup \Delta^+[\Psi](u)$;
end
return $\langle \Delta^-(u), \Delta^+(u) \rangle$;

5.4 Correctness and Minimality of the Incremental Formulation

This section states a series of results showing that the incremental formulation of changeset computation preserves the semantic effect of the original framework. Specifically, the results show that applying the incremental changeset to the pre-update RDF state yields exactly the RDF state of the view after the update. The section also proves that the changesets satisfy a minimality criteria. Appendix A contains the proofs.

In what follows, let $\mathcal{W}$= (V, $\boldsymbol{S}$, M) be an RDB2RDF view, $u = (D, I)$ be an update on a relation R, σ_0 and σ_1 be the database states before and after applying u, and $W_1 = RDF\_State(\mathcal{W}, \sigma_1)$ and $W_0 = RDF\_State(\mathcal{W}, \sigma_0)$ be the view states before and after applying u. Let $\Delta^-(u)$ and $\Delta^+(u)$ be the deletion and insertion components defined as in Section 5.

Let σ be a relational database state. Recall that $\mathcal{P}(\mathcal{W})$ denotes the set of pivot relations of $\mathcal{W}$. By Definition 3, the RDF state of $\mathcal{W}$ under σ is given by:

$$RDF\_State(\mathcal{W}, \sigma) = \bigcup_{R^* \in \mathcal{P}(\mathcal{W})} \bigcup_{p^* \in R^*(\sigma)} RDF\_State[R^*](p^*, \sigma)$$

The first lemma shows that each quad has a unique derivation associated with a single pivot tuple and transformation rule.

Lemma 1 (Uniqueness of Quad Generation).
For every quad $(s, p, o, g) \in RDF\_State(\mathcal{W}, \sigma)$, there exists a unique transformation rule $\Psi \in M$ and a unique pivot tuple $p^* \in pivot(\Psi)(\sigma)$ such that

$$(s, p, o, g) \in RDF\_State[\Psi](p^*, \sigma)$$

The next two lemmas show that the incremental deletion and the incremental insertion components capture exactly the quads whose derivations are invalidated by deletions, or created by insertions.

Recall that a quad *q has a derivation under state σ* iff there is a rule Φ, with pivot relation T^*, and a pivot tuple $t^* \in T^*(\sigma)$ such that $q \in \Phi[r \leftarrow t^*](\sigma)$.

Lemma 2 (Characterization of the Deletion Component).

(a) $(For\ any\ quad\ q)((q \in W_0 \wedge q \notin \Delta^-(u)) \Rightarrow q \in W_1)$.
(b) $(For\ any\ quad\ q)(q \in W_1 \Rightarrow (q \notin \Delta^-(u) \vee q \in \Delta^+(u))$.

Lemma 3 (Characterization of the Insertion Component).
$\Delta^+(u)$ is the set of quads whose derivation becomes valid in σ_1 due to some tuple $i \in I$.

Theorem 1 states that that the insertion and deletion changesets are correct.

Theorem 1 (Correctness of the Incremental Changesets).

$$W_1 = \big(W_0 \setminus \Delta^-(u)\big) \cup \Delta^+(u)$$

Finally, Theorem 1 has a simple corollary. Let $u = (D, I)$ be an update. Recall that σ_0 and σ_1 be the database states before and after applying u, and $W_0 = RDF\_State(\mathcal{W}, \sigma_0)$ and $W_1 = RDF\_State(\mathcal{W}, \sigma_1)$ are the view states before and after applying u. Let σ_D be the database state after applying the deletions in D, but before applying the insertions in I. Let $W_D = RDF\_State(\mathcal{W}, \sigma_D)$ be the view state corresponding to W_D.

Corollary 1 (Minimality of the Incremental Changesets).

(a) $\Delta^-(u) = (W_0 \setminus W_D)$
(b) $(\Delta^+(u) \setminus W_D) = (W_1 \setminus W_D)$
(c) If $(\Delta^+(u) \cap W_D) = \emptyset$, then $\Delta^+(u) = (W_1 \setminus W_D)$

Corollary 1 suggests that the changesets are minimal in the following sense. Part (a) says that $\Delta^-(u)$ is exactly the set of quads that were removed from the initial view state W_0 as a result of the deletions in the update u (but before the insertions). Likewise, Part (b) states that $\Delta^+(u)$ is the set of quads that were inserted into the intermediate view state W_D, created after the deletions, as a result of the insertions in the update u. However, we have to ignore quads that are already in W_D, since we do not rule out stale insertions that lead to duplicates. Part (c) is a simple consequence of Part (b) and says that, if $\Delta^+(u)$ has no duplicates w.r.t. W_D, then $\Delta^+(u)$ is the minimum set of quads that must be inserted into W_D to create W_1.

5.5 Example of Computing $\Delta^-(u)$ and $\Delta^+(u)$

We illustrate the computation of the changeset $\langle \Delta^-(u), \Delta^+(u) \rangle$ for an update on relation `Track`, following the formulation of the previous sections. In accordance with the maintenance strategy adopted in the article, we first compute $\Delta^-(u)$ and then $\Delta^+(u)$.

Update. Consider the update

```
UPDATE Track SET cid = 'c1' WHERE mid = 'm1';
```

From the database state in Figure 2, this update affects two tuples of `Track`. Hence, $u = (D, I)$, with $D = t_1, t_2$ and $I = t'_1, t'_2$, where

$$t_1 = (tid = t1, mid = m1, name = \text{"This Girl"}, cid = c2),$$
$$t_2 = (tid = t2, mid = m1, name = \text{"Don't You Know"}, cid = c3)$$
$$t'_1 = (tid = t1, mid = m1, name = \text{"This Girl"}, cid = c1),$$
$$t'_2 = (tid = t2, mid = m1, name = \text{"Don't You Know"}, cid = c1)$$

Relevant Rules. The transformation rules relevant to `Track` are:

$$Relev(\texttt{Track}) = \{\Psi_5, \Psi_7, \Psi_8, \Psi_{12}\}.$$

Among them, Ψ_5 and Ψ_{12} are pivot-relevant, whereas Ψ_7 and Ψ_8 are relation-relevant.

Phase 1: Computing $\Delta^-(u)$. By Section 6.2,

$$\Delta^-(u) = \bigcup_{\Psi \in Relev(\texttt{Track})} \Delta^-[\Psi](u), \qquad \Delta^-[\Psi](u) = \Delta^-_{pivot}[\Psi](u) \cup \Delta^-_{rel}[\Psi](u).$$

Pivot-relevant rules. For Ψ_5 and Ψ_{12}, the deleted tuples in D are themselves pivot tuples. Thus:

$$\Delta^-[\Psi_5](u) = \{(mbz : t1,\ rdf : type,\ mo : Track,\ \Psi_5),\\ (mbz : t2,\ rdf : type,\ mo : Track,\ \Psi_5)\}$$

and

$$\Delta^-[\Psi_{12}](u) = \{(mbz : t1,\ dc : title,\ \text{"This Girl"},\ \Psi_{12}),\\ (mbz : t2,\ dc : title,\ \text{"Don't You Know"},\ \Psi_{12})\}$$

Relation-relevant rule Ψ_7. Rule Ψ_7 is:

$$\Psi_7 :\ foaf{:}made(s, g) \leftarrow Artist(r), hasURI(mbz{:}, r.gid, s), fk1(r, r_1),\\ fk2(r_1, r_2), fk3(r_2, r_3), Track(r_3), hasURI(mbz{:}, r_3.tid, g)$$

Its pivot relation is `Artist`. Since `Track` occurs only once in $path(\Psi_7)$, the affected pivot tuples for the deletion phase are:

$$A^-[\Psi_7](u) = \{p^* \in Artist(\sigma_1) \mid \exists d \in D,\ \exists (s, p, o) \in \Psi_7[r \leftarrow p^*,\ r_3 \leftarrow d](\sigma_1)\}$$

Evaluating Ψ_7 with the deleted tuples yields:

$$t_1 \text{ with } cid = c2 \Rightarrow \text{artists } a2, a3, \qquad t_2 \text{ with } cid = c3 \Rightarrow \text{artist } a1$$

Hence,

$$A^-[\Psi_7](u) = \{a1, a2, a3\}$$

Now, for each $p^* \in A^-[\Psi_7](u)$, let

$$S_1[\Psi_7](p^*) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0,\ s = URI(p^*),\ g = URI(\Psi_7)\}$$

and

$$S_2[\Psi_7](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi_7[r \leftarrow p^*](\sigma_1),\ g = URI(\Psi_7)\}$$

Then:

$$\Delta^{-}_{rel}[\Psi_7](u) = \bigcup_{p^* \in A^{-}[\Psi_7](u)} (S_1[\Psi_7](p^*) \setminus S_2[\Psi_7](p^*))$$

Before the update, Ψ_7 generates:

$$(mbz : ga1,\ foaf{:}made,\ mbz : t2,\ \Psi_7)$$
$$(mbz : ga2,\ foaf{:}made,\ mbz : t1,\ \Psi_7)$$
$$(mbz : ga3,\ foaf{:}made,\ mbz : t1,\ \Psi_7)$$

After the update, Ψ_7 generates:

$$(mbz : ga1,\ foaf{:}made,\ mbz : t1,\ \Psi_7)$$
$$(mbz : ga1,\ foaf{:}made,\ mbz : t2,\ \Psi_7)$$
$$(mbz : ga2,\ foaf{:}made,\ mbz : t1,\ \Psi_7)$$
$$(mbz : ga2,\ foaf{:}made,\ mbz : t2,\ \Psi_7)$$

Therefore,

$$\Delta^{-}[\Psi_7](u) = \{(mbz : ga3,\ foaf{:}made,\ mbz : t1,\ \Psi_7)\}$$

Relation-relevant rule Ψ_8. Rule Ψ_8 is:

$$\Psi_8 :\ mo{:}track(s, g) \leftarrow Medium(r), hasURI(mbz{:}, r.mid, s), fk4(r, f), Track(f), hasURI(mbz{:}, f.tid, g)$$

Its pivot relation is `Medium`. Since `Track` occurs only once in $path(\Psi_8)$, the affected pivot tuples for the deletion phase are:

$$A^{-}[\Psi_8](u) = \{p^* \in Medium(\sigma_1) \mid \exists d \in D,\ \exists (s, p, o) \in \Psi_8[r \leftarrow p^*,\ f \leftarrow d](\sigma_1)\}$$

Because both deleted tuples satisfy $mid = m1$, we obtain:

$$A^{-}[\Psi_8](u) = \{m1\}$$

However, the RDF contribution of $m1$ under Ψ_8 is unchanged after the update, since $m1$ remains connected to the same track identifiers $t1$ and $t2$. Hence:

$$\Delta^{-}[\Psi_8](u) = \varnothing$$

Final deletion set. Collecting the contributions:

$$\Delta^{-}(u) = \Delta^{-}[\Psi_5](u) \cup \Delta^{-}[\Psi_{12}](u) \cup \Delta^{-}[\Psi_7](u) \cup \Delta^{-}[\Psi_8](u)$$

that is,

$$\begin{aligned}\Delta^{-}(u) = \{&(mbz : t1,\ rdf : type,\ mo : Track,\ \Psi_5),\\ &(mbz : t2,\ rdf : type,\ mo : Track,\ \Psi_5),\\ &(mbz : t1,\ dc : title,\ \text{"This Girl"},\ \Psi_{12}),\\ &(mbz : t2,\ dc : title,\ \text{"Don't You Know"},\ \Psi_{12}),\\ &(mbz : ga3,\ foaf{:}made,\ mbz : t1,\ \Psi_7)\}\end{aligned}$$

Phase 2: Computing $\Delta^+(u)$. By Section 6.3,

$$\Delta^+(u) = \bigcup_{\Psi \in Relev(\texttt{Track})} \Delta^+[\Psi](u), \qquad \Delta^+[\Psi](u) = \Delta^+_{pivot}[\Psi](u) \cup \Delta^+_{rel}[\Psi](u).$$

Pivot-relevant rules. For Ψ_5 and Ψ_{12}, the inserted tuples in I are pivot tuples. Thus:

$$\Delta^+[\Psi_5](u) = \{(mbz : t1,\ rdf : type,\ mo : Track,\ \Psi_5),\\ (mbz : t2,\ rdf : type,\ mo : Track,\ \Psi_5)\}$$

and

$$\Delta^+[\Psi_{12}](u) = \{(mbz : t1,\ dc : title,\ \text{"This Girl"},\ \Psi_{12}),\\ (mbz : t2,\ dc : title,\ \text{"Don't You Know"},\ \Psi_{12})\}$$

Relation-relevant rule Ψ_7. For the insertion phase, the affected pivot tuples are:

$$A^+[\Psi_7](u) = \{p^* \in Artist(\sigma_1) \mid \exists i \in I,\ \exists (s,p,o) \in \Psi_7[r \leftarrow p^*,\ r_3 \leftarrow i](\sigma_1)\}.$$

Evaluating Ψ_7 with the inserted tuples yields:

$$t'_1 \text{ with } cid = c1 \Rightarrow \text{artists } a1, a2, \qquad t'_2 \text{ with } cid = c1 \Rightarrow \text{artists } a1, a2.$$

Hence,

$$A^+[\Psi_7](u) = \{a1, a2\}.$$

For each $p^* \in A^+[\Psi_7](u)$, let

$$S1[\Psi_7](p^*) = \{(s,p,o,g) \mid (s,p,o,g) \in W_0,\ s = URI(p^*),\ g = URI(\Psi_7)\}$$

and

$$S2[\Psi_7](p^*) = \{(s,p,o,g) \mid (s,p,o) \in \Psi_7[r \leftarrow p^*](\sigma_1),\ g = URI(\Psi_7)\}.$$

Then:

$$\Delta^+_{rel}[\Psi_7](u) = \bigcup_{p^* \in A^+[\Psi_7](u)} \left(S2[\Psi_7](p^*) \setminus S1[\Psi_7](p^*)\right).$$

Therefore,

$$\Delta^+[\Psi_7](u) = \{(mbz : ga1,\ foaf{:}made,\ mbz : t1,\ \Psi_7),\\ (mbz : ga2,\ foaf{:}made,\ mbz : t2,\ \Psi_7)\}$$

Relation-relevant rule Ψ_8. For the insertion phase, the affected pivot tuples are:

$$A^+[\Psi_8](u) = \{p^* \in Medium(\sigma_1) \mid \exists i \in I,\ \exists (s,p,o) \in \Psi_8[r \leftarrow p^*,\ f \leftarrow i](\sigma_1)\}$$

Since both inserted tuples still satisfy $mid = m1$, we obtain:

$$A^+[\Psi_8](u) = \{m1\}$$

Again, the RDF contribution of $m1$ under Ψ_8 does not change, and hence:

$$\Delta^+[\Psi_8](u) = \varnothing$$

Final insertion set. Collecting the contributions:

$$\Delta^+(u) = \Delta^+[\Psi_5](u) \cup \Delta^+[\Psi_{12}](u) \cup \Delta^+[\Psi_7](u) \cup \Delta^+[\Psi_8](u)$$

that is,

$$\begin{aligned}\Delta^+(u) = \{&(mbz:t1,\ rdf:type,\ mo:Track,\ \Psi_5),\\ &(mbz:t2,\ rdf:type,\ mo:Track,\ \Psi_5),\\ &(mbz:t1,\ dc:title,\ \text{"This Girl"},\ \Psi_{12}),\\ &(mbz:t2,\ dc:title,\ \text{"Don't You Know"},\ \Psi_{12}),\\ &(mbz:ga1,\ foaf{:}made,\ mbz:t1,\ \Psi_7),\\ &(mbz:ga2,\ foaf{:}made,\ mbz:t2,\ \Psi_7)\}\end{aligned}$$

Discussion. The computation makes explicit that the affected sets differ between the deletion and insertion phases for Ψ_7:

$$A^-[\Psi_7](u) = \{a1, a2, a3\}$$
$$A^+[\Psi_7](u) = \{a1, a2\}$$

whereas for Ψ_8 both phases affect the same pivot tuple,

$$A^-[\Psi_8](u) = A^+[\Psi_8](u) = \{m1\}$$

but no net change is produced. Thus, the only effective semantic change in the example is the replacement of the obsolete `foaf:made` assertion for $a3$ by the new assertions induced by the reassignment of `cid`.

5.6 LLM-Compilability as a Consequence of Correctness and Structure

Beyond semantic correctness, the proposed framework exhibits structural properties that make it inherently amenable to LLM-based synthesis.

The correctness result (Theorem 1) establishes that the incremental changeset fully captures the effect of an update on the RDF view. This provides a sound

and complete specification of the update semantics. However, correctness alone does not guarantee that such a specification can be effectively translated into executable procedures.

A key observation of this work is that LLM-compilability arises from the combination of correctness with specific structural properties of the formalization.

First, the incremental formulation is **compositional**: both $\Delta^{-inc}(u)$ and $\Delta^{+inc}(u)$ are defined as unions of pivot-based and path-based contributions. This enables the synthesis process to decompose the problem into smaller, independently computable units.

Second, the framework exhibits **locality**: the effect of an update is determined solely by the modified relation R and its participation in transformation rules. This eliminates the need for global reasoning over the entire database state.

Third, and most critically, the framework ensures **deterministic derivations**. By Lemma 1, each RDF quad is associated with a unique pivot tuple and transformation rule. This property removes ambiguity in the mapping between input tuples and output quads, which is essential for reliable code generation.

These properties collectively enable a systematic translation from formal definitions to executable artifacts. In particular, large language models can (i) derive abstract algorithms that operationalize the definitions, and (ii) generate concrete trigger implementations that compute the incremental changeset.

This leads to a broader methodological insight: formal data management frameworks can be designed not only for semantic correctness, but also for LLM-compilability. In this perspective, the formal specification serves as a compilation contract, ensuring that any generated implementation preserves the intended semantics.

To the best of our knowledge, this work is among the first to explicitly connect formal correctness, structural properties of specifications, and LLM-based compilation in the context of data management.

A central design goal of the proposed formulation is to enable *LLM-compilability*, i.e., the ability to systematically translate formal definitions into correct and minimal executable implementations using large language models.

The refined formulation of the incremental deletion component achieves this by decomposing the computation into *independent contributions per pivot tuple*. More precisely, instead of requiring a global evaluation of a transformation rule over the entire database state, the computation is structured around the set of relevant pivot tuples:

$$A = \{p^* \in R^*(\sigma_1) \mid \exists d \in D \text{ such that } p^* \in P[\Psi](d; \sigma_1)\}\,.$$

For each $p^* \in A$, the incremental deletion contribution is computed locally as $\Delta^-[\Psi](p^*, \sigma_1)$, and the final result is obtained by aggregation.

This decomposition has an important implications for LLM-based synthesis: Each contribution $\Delta^-[\Psi](p^*, \sigma_1)$ can be computed independently, given a binding for p^* and the deleted tuples $d \in D$. This avoids the need for global reasoning over all tuples in the pivot relation and aligns with the localized inference patterns that LLMs handle more reliably.

6 Implementation

This section describes an implementation of an incremental view maintenance tool for RBD2RDF views specified by R2RML statements. If first describes an architecture for an incremental view maintenance tool, featuring AFTER triggers to capture the database updates and a specialized worker to maintain the view. Finally, it discusses the key question of compiling the AFTER triggers from the R2RML statements, using transformation rules as an intermediate stage.

6.1 An Architecture for a Concrete Implementation

Algorithm 1 defines how to compute the changesets $\Delta^-(u)$ and $\Delta^+(u)$, but it must be re-engineered to separate accessing the post-update database state σ_1 from accessing the pre-update view state W_0, since the database and the view are typically in separate environments. This section then explores an architecture, shown in Figure 4, that addresses this problem, assuming that the database is stored in PostgreSQL and the view in GraphDB.

The first step is to rewrite the definition of $\Delta^-(u)$ as follows. First define:

$$
\begin{aligned}
S_1[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} S_1[\Psi](p^*) \\
&= \bigcup_{p^* \in A^-[\Psi](u)} \{(s,p,o,g) \mid (s,p,o,g) \in W_0,\ s = URI(p^*),\ g = URI(\Psi)\} \\
S_2[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} S_2[\Psi](p^*) \\
&= \bigcup_{p^* \in A^-[\Psi](u)} \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1),\ g = URI(\Psi)\}
\end{aligned}
$$

The following proposition shows that $\Delta^-_{rel}[\Psi](u)$ can be redefined using $S_1[\Psi](u)$, computed on W_0 and $A^-[\Psi](u)$, and $S_2[\Psi](u)$, computed on σ_1 and $A^-[\Psi](u)$.

Proposition 1. *Let $\mathcal{W}$= (V, **S**, M) be an RDB2RDF view. Assume that $\mathcal{W}$ is entity-preserving. Then,*

$$\Delta^-_{rel}[\Psi](u) = S_1[\Psi](u) \setminus S_2[\Psi](u)$$

(The proof can be found in Appendix A).

Recall that:

$$\begin{aligned}
\Delta^{-}_{pivot}[\Psi](u) &= \bigcup_{d \in D} \Delta^{-}_{pivot}[\Psi](d) \\
&= \bigcup_{d \in D} \{(s,p,o,g) \mid (s,p,o,g) \in W_0,\ s = URI(d),\ g = URI(\Psi)\} \\
A^{-}[\Psi](u) &= \{p^* \in R^*(\sigma_1) \mid \exists d \in D,\ \exists r_k \in VarR(\Psi), \\
&\qquad \exists (s,p,o) \in \Psi[r^* \leftarrow p^*,\ r_k \leftarrow d](\sigma_1[R \leftarrow ((R \cup D) \setminus I]))\} \\
S_1[\Psi](u) &= \bigcup_{p^* \in A^{-}[\Psi](u)} \{(s,p,o,g) \mid (s,p,o,g) \in W_0,\ s = URI(p^*),\ g = URI(\Psi)\} \\
S_2[\Psi](u) &= \bigcup_{p^* \in A^{-}[\Psi](u)} \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1),\ g = URI(\Psi)\} \\
\Delta^{-}_{rel}[\Psi](u) &= S_1[\Psi](u) \setminus S_2[\Psi](u) \\
\Delta^{-}[\Psi](u) &= \Delta^{-}_{pivot}[\Psi](u) \cup \Delta^{-}_{rel}[\Psi](u)
\end{aligned}$$

and

$$\begin{aligned}
\Delta^{+}_{pivot}[\Psi](u) &= \bigcup_{i \in I} \Delta^{+}_{pivot}[\Psi](i) \\
&= \bigcup_{i \in I} \{(s,p,o,g) \mid (s,p,o) \in \Psi[r \leftarrow i](\sigma_1),\ g = URI(\Psi)\} \\
A^{+}[\Psi](u) &= \{p^* \in R^*(\sigma_1) \mid \exists i \in I,\ \exists r_k \in VarR(\Psi), \\
&\qquad \exists (s,p,o) \in \Psi[r^* \leftarrow p^*,\ r_k \leftarrow i](\sigma_1)\} \\
\Delta^{+}_{rel}[\Psi](u) &= \bigcup_{p^* \in A^{+}[\Psi](u)} \Delta^{+}_{rel}[\Psi](p^*) \\
&= \bigcup_{p^* \in A^{+}[\Psi](u)} \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1),\ g = URI(\Psi)\} \\
\Delta^{+}[\Psi](u) &= \Delta^{+}_{pivot}[\Psi](u) \cup \Delta^{+}_{rel}[\Psi](u)
\end{aligned}$$

Let $u = (D, I)$ be an update, W_0 be the pre-update view state, and σ_1 be the post-update database state. Then, note that:

- $\Delta^{-}_{pivot}[\Psi](u)$: defined using W_0 and u.
- $A^{-}[\Psi](u)$: defined using σ_1 and u.
- $S_1[\Psi](u)$: defined using W_0 and $A^{-}[\Psi](u)$.
- $S_2[\Psi](u)$: defined using σ_1 and $A^{-}[\Psi](u)$.
- $\Delta^{-}[\Psi](u)$: defined using W_0, σ_1, and u.
- $\Delta^{+}_{pivot}[\Psi](u)$: defined using σ_1 and u.
- $A^{+}[\Psi](u)$: defined using σ_1 and u.
- $\Delta^{+}_{rel}[\Psi](u)$: defined using σ_1 and $A^{+}[\Psi](u)$.
- $\Delta^{+}[\Psi](u)$: defined using σ_1 and u.

Now, the architecture decomposes the computation of $\Delta^-[\Psi](u)$ and $\Delta^+[\Psi](u)$, according to the use of W_0 and σ_1. In more detail, instead of directly computing and applying the changeset inside the database transaction, a trigger captures a logical snapshot of the update event, including the transition tables D and I, and computes the affected tuples for deletion $A^-[\Psi](u)$, the affected tuples for insertion $A^+[\Psi](u)$, the post-update contributions $S_2[\Psi](u)$ to $\Delta^-(u)$, and the insertion changeset $\Delta^+(u)$. This information is then stored in a maintenance queue. The triggers are generated by translating the pseudo-code in Algorithm 1 into SQL code, except for Lines 9 and 27, which depend on the pre-update view state W_0, using an LLM (Table 6 contains URLs with examples of the generated code).

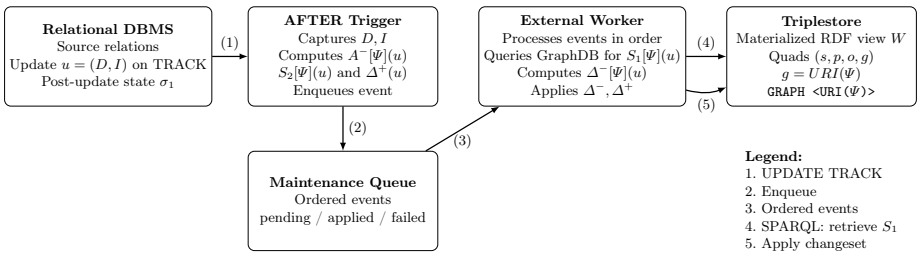


Fig. 4. Asynchronous architecture for incremental maintenance of a materialized RDB2RDF view. AFTER triggers capture the transition tables D and I, and compute $A^-[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta^+[\Psi](u)$ (which requires computing $A^+[\Psi](u)$). An external worker processes queued events in commit order, retrieves $\Delta^-_{pivot}[\Psi](u)$ and $S_1[\Psi](u)$ from the triplestore, computes $\Delta^-[\Psi](u)$, and updates the RDF materialization.

The architecture combines AFTER statement-level triggers with an asynchronous maintenance worker. Instead of directly computing and applying the changeset inside a database transaction, a trigger captures a logical snapshot of the update event, i.e., the transition tables D and I, and computes the affected tuples for deletion $A^-[\Psi](u)$, the post-update contributions $S_2[\Psi](u)$ to $\Delta^-(u)$, and the insertion changeset $\Delta^+(u)$ (which requires computing the affected tuples for insertion $A^+[\Psi](u)$). This information is then stored in a maintenance queue. The AFTER trigger for a relation R calls the function Compute_Changeset_R, shown in Algorithm 2, to compute $A^-[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta^+[\Psi](u)$ and update the maintenance queue.

Table 6 contains URLs with examples of the LLM-generated trigger code, and Algorithm 2 summarizes the pseudocode.

An external worker processes queued events sequentially, preserving the update commit order. For each event, the worker first queries the triplestore to retrieve from the RDF view state W_0 the corresponding deletion contribution of deleted tuples, $\Delta^-_{pivot}[\Psi](u)$, and the pre-update RDF contributions, $S_1[\Psi](u)$. Then, it

Algorithm 2: Function COMPUTE_CHANGESET_R for the Asynchronous Architecture

```
   // Event-level queue initialization:
1. Insert one row into rdf_maintenance_queue for the statement-level update
     u = (D, I).
   // Rule-level computation and publication:
2. foreach Ψ ∈ Relev(R) do
       // Execute the following two steps sequentially:
       // STEP 1 - Incremental Maintenance Computation for Ψ
3.     Δ+_pivot[Ψ](u) ← ∅
4.     Δ+_rel[Ψ](u) ← ∅
5.     if pivot(Ψ) = R then
6.         Compute Δ+_pivot[Ψ](u)
7.     end
8.     if R ∈ Relations(path(Ψ)) then
9.         Compute A−[Ψ](u), A+[Ψ](u), S2[Ψ](u), and Δ+_rel[Ψ](u)
10.    end
11.    Δ+[Ψ](u) ← Δ+_pivot[Ψ](u) ∪ Δ+_rel[Ψ](u)
       // STEP 2 - Rule Contribution Publication
12.    Insert one row into rdf_rule_contribution for Ψ
13.            with A−[Ψ](u), S2[Ψ](u), and Δ+[Ψ](u),
14.            linked to the event_id created in rdf_maintenance_queue.
15. end
```

computes the deletion contribution using the set difference $(S_1[\Psi](u) \setminus S_2[\Psi](u))$, applies $\Delta^-(u)$ to the view, and then applies $\Delta^+(u)$.

This architecture offers several advantages. First, it avoids executing SPARQL queries inside relational transactions, preventing triplestore latency or temporary unavailability from blocking database updates. Second, it preserves correctness by storing the relational context of the update at commit time, ensuring that subsequent database updates do not affect the computation of the changeset. Third, the queue-based design improves scalability, fault tolerance, and recoverability, since maintenance events can be retried or processed asynchronously without impacting transactional workloads.

Finally, pseudocode for the components of the architecture, generated by ChatGPT, can be found at:

- Architecture: `https://chatgpt.com/s/t_69f3785b8b3c81918e736190feccf012`
- Trigger: `https://chatgpt.com/s/t_69f65bcaf4788191b549fbab333b13c4`
- Worker: `https://chatgpt.com/s/t_69f5233aacf8819188f012286ecf2e07`

6.2 Compilation of R2RML Statements into AFTER Triggers

Given an RDB2RML view, specified by R2RML statements, the compilation of the AFTER triggers from the R2RML statements proceeds in two steps, using transformation rules (TRs) as an intermediate stage.

The first step compiles the R2RML statements into TRs, using an LLM, guided by a prompt (available at the supplemental material) that accesses this paper and the R2RML statements as attachments. The prompt also guides the LLM to analyze the R2RML statements, check if they define an entity-preserving view, and propose changes to the view definition, if the check fails.

The second step then compiles the TRs into AFTER triggers, with the separation of concerns discussed in Section 6.1. That is, each trigger captures a snapshot of the update event on a relation R (the transition tables D and I), and computes the affected tuples for deletion $A^{-}[\Psi](u)$, the post-update contributions $S_2[\Psi](u)$ to $\Delta^{-}(u)$, and the insertion changeset $\Delta^{+}(u)$ (which requires computing the affected tuples for insertion $A^{+}[\Psi](u)$).

Again, the compilation step is performed by an LLM, guided by a prompt that accesses this paper, the TRs generated by the first step, the view ontology, and the relational schema as attachments. Table 6 contains URLs with examples of the LLM-generated trigger code.

7 Experiments

7.1 Experimental Setup

Data and LLMs. The experiments were based on:

- *Relational Database*: a local copy of the *MusicBrainz* database.
- *LinkedBrainz RDF View*: an RDF view materialized in a triplestore.
- *Selected relations*: the relations `TRACK`, `ARTIST`, and `MEDIUM` were chosen because they occur in transformation rules with complex paths.
- *Selected updates*: a list with one insertion, one deletion, and one update per selected relation, along with their expected changesets.
- *Selected LLMs*: gpt-5.3-chat, DeepSeek V3.2, and Qwen3.6-Plus.

MusicBrainz[2] is a large and actively maintained open music knowledge base operated by the MetaBrainz Foundation. The relational database schema comprises 106 relations, with data about artists, release groups, etc. The database is implemented in PostgreSQL and continuously publishes incremental database updates through its replication mechanism, providing a realistic scenario of high-frequency relational changes. This makes the case study particularly suitable for evaluating incremental maintenance techniques for materialized RDB2RDF views.

[2] Available at `https://musicbrainz.org`

The LinkedBrainz RDF View[3] uses the Music Ontology vocabulary and is specified in R2RML through 57 TriplesMaps[4]. Although MusicBrainz already distributes incremental relational updates through replication packets, maintaining a materialized view over MusicBrainz still requires computing changesets that reflect the impact of relational updates on the RDF materialized view.

Experiment stages[5]. The experiments were run in two stages:

- Stage 1 prompted each selected LLM to generate all AFTER triggers, as described in Section 6.2. For each combination of selected LLM and selected relation, the resulting code was manually evaluated according to the criteria in Table 5.
- For each combination of selected LLM and selected relation, Stage 2 ran the code generated in Stage 1 with the selected updates. The resulting changesets were manually evaluated according to the criterion in Table 5.

Note that, to avoid a combinatorial explosion, the (manual) evaluation of the results was constrained to use the selected LLMs and to cover only the selected relations and updates.

Stage 1 called each LLM through its free-tier chat interface: gpt-5.3-chat and DeepSeek V3.2 in "instant" mode, and Qwen3.6-Plus in "auto" mode.

Evaluation criteria. Table 5 lists the evaluation criteria. The criteria for Stage 1 assess whether an LLM correctly interprets the formal definitions and generates code to compute the changesets $\Delta^-(u)$ and $\Delta^+(u)$. The criterion for Stage 2 compares, for each selected update, the changesets obtained by running the code against the database with the expected changesets.

Table 5. Evaluation Criteria.

Stage	Dimension	Description
1	TR Identification	Does the model correctly identify the relevant transformation rules?
	Computation of $\Delta^-(u)$	Does the model correctly generate the code for $\Delta^-(u)$?
	Computation of $\Delta^+(u)$	Does the model correctly generate the code for $\Delta^+(u)$?
	Code Minimality	Is the generated code concise and free of unnecessary operations?
	Hallucinations	Does the model introduce unsupported assumptions or irrelevant joins?
2	Execution Correctness	Does the code compute the expected changesets for each update?

[3] Available at `https://musicbrainz.org/doc/LinkedBrainz/Software_Approach`
[4] Available at `https://docs.openlinksw.com/virtuoso/rdfviewsbusintmbr/`
[5] The code, database, and RDF view adopted in the experiments are available at `https://anonymous.4open.science/r/llm-assisted-compilation-F7B8`

7.2 Results

The results are organized in four tables, as follows:

- Table 6: contains the session URLs with the AFTER Triggers generated.
- Tables 7, 8, and 9: summarize the manual evaluation for each selected relation.

To summarize, the results suggest that:

- gpt-5.3-chat: generated correct and optimized code in all cases.
- DeepSeek V3.2: generated correct code, but not optimized in some cases.
- Qwen3.6-Plus: generated correct and concise code in some cases.

Table 6. Session URLs with the AFTER Triggers generated for each selected relation.

Modelo	Relation	Generated AFTER Trigger – Session URL
gpt-5.3-chat	TRACK	https://chatgpt.com/s/t_69f670cfb1d0819188e313d12e743040
	ARTIST	
	MEDIUM	
DeepSeek V3.2	TRACK	
	ARTIST	
	MEDIUM	
Qwen3.6-Plus	TRACK	
	ARTIST	
	MEDIUM	

Table 7. Evaluation for relation TRACK.

Stage	Evaluation Criteria	Model	Result
1	Transformation rule Identification	gpt-5.3-chat	identified all relevant rules
		DeepSeek V3.2	identified all relevant rules
		Qwen3.6-Plus	identified all relevant rules
1	Computation of $\Delta^{-}(u)$	gpt-5.3-chat	correctly produced all relational joins
		DeepSeek V3.2	incorrectly produced some relational joins
		Qwen3.6-Plus	distinguished two formal cases
1	Computation of $\Delta^{+}(u)$	gpt-5.3-chat	correctly computed RDF_STATE
		DeepSeek V3.2	generated only partial RDF triples
		Qwen3.6-Plus	generated only partial RDF triples
1	Code minimality	gpt-5.3-chat	produced minimal code
		DeepSeek V3.2	introduced redundant intermediate queries
		Qwen3.6-Plus	relatively clean and concise
1	Hallucinations	gpt-5.3-chat	did not hallucinate
		DeepSeek V3.2	generated unsupported logic
		Qwen3.6-Plus	limited, but one unsupported simplification
2	Execution correctness	gpt-5.3-chat	
		DeepSeek V3.2	
		Qwen3.6-Plus	

Table 8. Evaluation for relation ARTIST.

Stage	Evaluation Criteria	Model	Result
1	Transformation rule Identification	gpt-5.3-chat	identified all relevant rules
		DeepSeek V3.2	identified all relevant rules
		Qwen3.6-Plus	identified all relevant rules
1	Computation of $\Delta^-(u)$	gpt-5.3-chat	correctly produced all relational joins
		DeepSeek V3.2	incorrectly produced some relational joins
		Qwen3.6-Plus	distinguished two formal cases
1	Computation of $\Delta^+(u)$	gpt-5.3-chat	correctly computed RDF_STATE
		DeepSeek V3.2	generated only partial RDF triples
		Qwen3.6-Plus	generated only partial RDF triples
1	Code minimality	gpt-5.3-chat	produced minimal code
		DeepSeek V3.2	introduced redundant intermediate queries
		Qwen3.6-Plus	relatively clean and concise
1	Hallucinations	gpt-5.3-chat	did not hallucinate
		DeepSeek V3.2	generated unsupported logic
		Qwen3.6-Plus	limited, but one unsupported simplification
2	Execution correctness	gpt-5.3-chat	
		DeepSeek V3.2	
		Qwen3.6-Plus	

Table 9. Evaluation for relation MEDIUM.

Stage	Evaluation Criteria	Model	Result
1	Transformation rule Identification	gpt-5.3-chat	identified all relevant rules
		DeepSeek V3.2	identified all relevant rules
		Qwen3.6-Plus	identified all relevant rules
1	Computation of $\Delta^-(u)$	gpt-5.3-chat	correctly produced all relational joins
		DeepSeek V3.2	incorrectly produced some relational joins
		Qwen3.6-Plus	distinguished two formal cases
1	Computation of $\Delta^+(u)$	gpt-5.3-chat	correctly computed RDF_STATE
		DeepSeek V3.2	generated only partial RDF triples
		Qwen3.6-Plus	generated only partial RDF triples
1	Code minimality	gpt-5.3-chat	produced minimal code
		DeepSeek V3.2	introduced redundant intermediate queries
		Qwen3.6-Plus	relatively clean and concise
1	Hallucinations	gpt-5.3-chat	did not hallucinate
		DeepSeek V3.2	generated unsupported logic
		Qwen3.6-Plus	limited, but one unsupported simplification
2	Execution correctness	gpt-5.3-chat	
		DeepSeek V3.2	
		Qwen3.6-Plus	

8 Conclusions

The first contribution of this paper was a formal definition of the deletion and insertion components of RDB2RDF changesets, designed with LLM-compilation in mind. For a transformation rule without multiple occurrences of the same relation, the definitions of both the deletion and insertion components use only the post-update state, transition tables, and the pre-update view state. For a transformation rule with multiple occurrences of the same relation R, the definitions capture the impact of deletions and insertions on R directly in the evaluation of the transformation rule, still without using the pre-update database state.

The second contribution was to describe an implementation of an incremental view maintenance tool for RBD2RDF views specified by R2RML statements. The tool is based on a generic architecture, featuring AFTER triggers to capture the database updates and a specialized worker to maintain the view. The implementation addressed the key question of compiling the AFTER triggers from the R2RML statements, using transformation rules as an intermediate stage.

The third contribution was to carry out an experiment with a real-world database and an RDB2RDF view, specified by R2RML statements, illustrating that an LLM, guided by appropriate prompts and accessing the formal definitions, can generate code that computes and publishes the changesets.

In fact, the construction of the prompts co-evolved with the formal definitions in the following sense. The experiment started with simple prompts, with the original version of the full paper as an attachment. An analysis of the results indicated that some errors were due to too simple prompts, on the one hand, and to missing definitions in the paper, on the other hand. The analysis also pointed out that the formal definitions could be changed to induce simpler database code. The prompts and the formal definitions were altered accordingly, and the tests were repeated until satisfactory results were achieved, as reported in the paper.

As future work, the pre-processing steps described in Section 6.2 will be re-engineered as a multi-agent system, with human in the loop, to facilitate compiling R2RML statements into AFTER triggers, and installing them in the source relational database.

Supplemental Material Statement. The code, prompts, database, and RDF view adopted in the experiments are available at
`https://anonymous.4open.science/r/llm-assisted-compilation-F7B8`

Declaration of use of Generative AI. In Section 5, ChatGPT was used to help check the definitions. In Section 6, ChatGPT was used to help define the architecture. In Section 7, three LLMs were utilized to generate code.

Appendix A. Proofs

In what follows, let $\mathcal{W}$= (V, $\boldsymbol{S}$, M) be an RDB2RDF view, $u = (D, I)$ be an update on a relation R, σ_0 and σ_1 be the database states before and after applying u, and $W_1 = RDF\_State(\mathcal{W}, \sigma_1)$ and $W_0 = RDF\_State(\mathcal{W}, \sigma_0)$ be the view states before and after applying u. Let $\Delta^-(u)$ and $\Delta^+(u)$ be the deletion and insertion components defined as in Section 5.

Let Ψ be a rule in M. Recall from Section 5.2 that

$$\Delta^-_{rel}[\Psi](p^*) = (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

Also recall that the overall incremental deletion contribution of a rule $\Psi \in \mathcal{M}$ for an update $u = (D, I)$ on a relation R, when Ψ is relation-relevant to R, is defined by aggregating the contributions of all affected pivot tuples:

$$\Delta^-_{rel}[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} \Delta^-_{rel}[\Psi](p^*)$$

Also recall that

$$S_1[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} S_1[\Psi](p^*)$$

$$S_2[\Psi](u) = \bigcup_{p^* \in A^-[\Psi](u)} S_2[\Psi](p^*)$$

Proposition 1. *Let $\mathcal{W}$ = (V, $\boldsymbol{S}$, M) be an RDB2RDF view. Assume that $\mathcal{W}$ is entity-preserving. Then,*

$$\Delta^-_{rel}[\Psi](u) = S_1[\Psi](u) \setminus S_2[\Psi](u)$$

Proof. Let p and q be two distinct pivot tuples of Ψ. Since, by assumption, $\mathcal{W}$ is entity-preserving, it follows that $URI(p) \neq UTI(q)$. Hence, from the definition of $S_1[\Psi](p)$ and $S_2[\Psi](q)$, it follows that, for $m, n \in \{1, 2\}$, with $m \neq n$:

$$S_m[\Psi](p) \cap S_n[\Psi](q) = \emptyset$$

which implies that

$$\begin{aligned}
\Delta^-_{rel}[\Psi](u) &= \bigcup_{p^* \in A^-[\Psi](u)} \Delta^-_{rel}[\Psi](p^*) \\
&= \bigcup_{p^* \in A^-[\Psi](u)} (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)) \\
&= (\bigcup_{p^* \in A^-[\Psi](u)} S_1[\Psi](p^*)) \setminus (\bigcup_{p^* \in A^-[\Psi](u)} S_2[\Psi](p^*)) \\
&= S_1[\Psi](u) \setminus S_2[\Psi](u)
\end{aligned}$$

□

Let σ be a relational database state. Recall that $\mathcal{P}(\mathcal{W})$ denotes the set of pivot relations of $\mathcal{W}$. By Definition 3, the RDF state of $\mathcal{W}$ under σ is given by:

$$RDF\_State(\mathcal{W}, \sigma) = \bigcup_{R^* \in \mathcal{P}(\mathcal{W})} \bigcup_{p^* \in R^*(\sigma)} RDF\_State[R^*](p^*, \sigma)$$

The first lemma shows that each quad has a unique derivation associated with a single pivot tuple and transformation rule.

Lemma 1 (Uniqueness of Quad Generation).
For every quad $(s, p, o, g) \in RDF\_State(\mathcal{W}, \sigma)$, there exists a unique transformation rule $\Psi \in M$ and a unique pivot tuple $p^* \in pivot(\Psi)(\sigma)$ such that

$$(s, p, o, g) \in RDF\_State[\Psi](p^*, \sigma)$$

Proof. By definition of $RDF\_State(\mathcal{W}, \sigma)$, every quad (s, p, o, g) is generated by applying some transformation rule $\Psi \in M$ to some pivot tuple $p^* \in pivot(\Psi)(\sigma)$. Thus, existence is immediate.

We now prove uniqueness.

Suppose that a quad (s, p, o, g) is generated by two pairs (Ψ_1, p_1) and (Ψ_2, p_2), i.e.,

$$(s, p, o, g) \in RDF\_State[\Psi_1](p_1, \sigma) \quad \text{and} \quad (s, p, o, g) \in RDF\_State[\Psi_2](p_2, \sigma)$$

We show that $\Psi_1 = \Psi_2$ and $p_1 = p_2$.

Assume $\Psi_1 \neq \Psi_2$.

By construction of RDF state, the provenance component g is defined as $g = URI(\Psi)$. Therefore, quads generated by different transformation rules have different provenance identifiers. Hence, (s, p, o, g) cannot be generated by both Ψ_1 and Ψ_2, which contradicts the assumption. Therefore, $\Psi_1 = \Psi_2$, which contradicts the assumption of the case.

Assume $\Psi_1 = \Psi_2 = \Psi$, but $p_1 \neq p_2$.

By the entity-preserving assumption, distinct pivot tuples are mapped to distinct RDF subjects. Therefore, all triples generated from p_1 have $URI(p_1)$ as subject, and all triples generated from p_2 have $URI(p_2)$ as subject, with

$$URI(p_1) \neq URI(p_2)$$

Thus, no triple (and hence no quad) can be generated from both p_1 and p_2. This contradicts the assumption that (s, p, o, g) belongs to both $RDF\_State[\Psi](p_1, \sigma)$ and $RDF\_State[\Psi](p_2, \sigma)$. Thus, $p_1 = p_2$, which contradicts the assumption of the case.

Therefore, $\Psi_1 = \Psi_2$ and $p_1 = p_2$, which proves uniqueness.

□

□

The next two lemmas show that the incremental deletion and the incremental insertion components capture the quads whose derivations are invalidated by deletions, or created by insertions.

Recall that a quad q *has a derivation under state* σ iff there is a rule Φ, with pivot relation T^*, and a pivot tuple $t^* \in T^*(\sigma)$ such that $q \in \Phi[r \leftarrow t^*](\sigma)$.

Lemma 2 (Characterization of the Deletion Component).

(a) $(\textit{For any quad } q)((q \in W_0 \wedge q \notin \Delta^-(u)) \Rightarrow q \in W_1)$.

(b) $(\textit{For any quad } q)(q \in W_1 \Rightarrow (q \notin \Delta^-(u) \vee q \in \Delta^+(u))$.

Proof

Part (a): We first note that Part (a) is equivalent to

$$(\text{For any quad } q)((q \in W_0 \wedge q \notin W_1) \Rightarrow q \in \Delta^-(u))$$

Assume that $q \in W_0$ and $q \notin W_1$. We prove that $q \in \Delta^-(u)$.

Since $q = (s, p, o, g) \in W_0$, there exist a transformation rule $\Psi \in M$ and a pivot tuple $p^* \in pivot(\Psi)(\sigma_0)$ such that

$$(s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_0) \quad \text{and} \quad g = URI(\Psi)$$

Let $u = (D, I)$ be the update over relation R. There are two cases to consider.

Case 1: $p^* \in D$.

Then, the definition of $\Delta^-_{pivot}[\Psi]$ in Section 4.2.1 applies to p^*:

$$\Delta^-_{pivot}[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o, g) \in W_0,\ s = URI(p^*),\ g = URI(\Psi)\}$$

Since $q = (s, p, o, g) \in W_0$, q is derived from p^* using Ψ in σ_0, and $p^* \in D$, it follows that:

$$q \in \Delta^-_{pivot}[\Psi](p^*) \subseteq \Delta^-(u)$$

Case 2: $p^* \notin D$.

Then, since $q \notin W_1$, there must be an occurrence of R in $path(\Psi)$ such that this occurrence is not the pivot relation of ψ (that is, the occurrence of R is not the first occurrence of a relation scheme in $path(\Psi)$) and this occurrence is affected by the deletions in $u = (D, I)$. Hence, ψ is relation-relevant to R with respect to $u = (D, I)$ and the definitions in Section 4.2.2 apply to R. Let R^* be the pivot relation of Ψ.

Since $q \notin W_1$, the pivot tuple p^* that generated q in σ_0 using Ψ is affected by the deletion of tuples in D, and therefore

$$p^* \in A^-[\Psi](u)$$

But, by assumption, $q \in W_0$ and q is derived using the pivot tuple p^* and the rule Ψ in σ_0. Hence, by definition of $S_1[\Psi](p^*)$, it follows that

$$q \in S_1[\Psi](p^*)$$

Also, by assumption, $q \notin W_1$. Hence, by definition of $S_2[\Psi](p^*)$, it follows that

$$q \notin S_2[\Psi](p^*)$$

Hence,

$$q \in (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

Therefore, for the pivot tuple p^*, by definition of $\Delta^-_{rel}[\Psi](p^*)$ in Section 4.2.2, it follows that:

$$q \in (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)) = \Delta^-_{rel}[\Psi](p^*)$$

Hence

$$q \in \Delta^-_{rel}[\Psi](u) \subseteq \Delta^-(u)$$

Thus, in both cases, if $q \in W_0$ and $q \notin W_1$, then $q \in \Delta^-(u)$.

Part (b): We first note that Part (b) is equivalent to

$$(\text{For any quad } q)((q \in \Delta^-(u) \wedge q \notin \Delta^+(u)) \Rightarrow q \notin W_1))$$

Assume that $q \in \Delta^-(u)$ and $q \notin \Delta^+(u)$. We prove that $q \notin W_1$.

By Section 4.1, the deletion component is defined as the union of the deletion contributions of all transformation rules relevant to R:

$$\Delta^-(u) = \bigcup_{\Psi \in Relev(R)} \Delta^-[\Psi](u)$$

Hence, there exists a transformation rule Ψ relevant to R such that

$$q \in \Delta^-[\Psi](u)$$

There are two cases to consider, according to the decomposition of Section 4.2:

$$\Delta^-[\Psi](u) = \Delta^-_{pivot}[\Psi](u) \cup \Delta^-_{rel}[\Psi](u)$$

Case 1: $q \in \Delta^-_{pivot}[\Psi](u)$.

In this case, Ψ is pivot-relevant to R, that is, $pivot(\Psi) = R$. Recall that $u = (D, I)$. By Section 4.2.1,

$$\Delta^-_{pivot}[\Psi](u) = \bigcup_{d \in D} \Delta^-_{pivot}[\Psi](d)$$

where each $\Delta^-_{pivot}[\Psi](d)$ is exactly the RDF contribution of a deleted pivot tuple $d \in D$ using Ψ in state σ_0.

By Lemma 1, the transformation rule Ψ and the pivot tuple $d \in D$ are the unique way of deriving q in σ_0. In particular, the subject of q is $URI(d)$. Hence, there would be no pivot tuple $t \in (R(\sigma_0) \setminus D)$ such that the subject of q is $URI(t)$.

By Lemma 3, $\Delta^+(u)$ is the set of quads whose derivation becomes valid in σ_1 due to some tuple $i \in I$. But $q \notin \Delta^+(u)$ by assumption. Hence, q has no derivation in σ_1 due to some tuple $i \in I$. In particular, there would be no pivot tuple $t \in I$ such that the subject of q is $URI(t)$.

Now, $R(\sigma_1) = ((R(\sigma_0) \setminus D) \cup I)$. Therefore, there would be no pivot tuple $t \in R(\sigma_1)$ such that the subject of q is $URI(t)$. Hence, q has no derivation in σ_1, that is, $q \notin W_1$.

Case 2: $q \in \Delta^-_{rel}[\Psi](u)$**.**

In this case, Ψ is relation-relevant to R, that is, $R \in Relations(path(\Psi))$. Let R^* be the pivot relation of Ψ. By Section 4.2.2, there exists an affected pivot tuple $p^* \in R^*(\sigma_0)$ such that

$$p^* \in A^-[\Psi](u)$$

and

$$q \in \Delta^-_{rel}[\Psi](p^*)$$

But, for relation-relevant rules, the deletion contribution of p^* is defined by

$$\Delta^-_{rel}[\Psi](p^*) = (S_1[\Psi](p^*) \setminus S_2[\Psi](p^*))$$

where

$$S_1[\Psi](p^*) = \{(s,p,o,g) \mid (s,p,o,g) \in W_0,\ s = URI(p^*),\ g = URI(\Psi)\}$$

$$S_2[\Psi](p^*) = \{(s,p,o,g) \mid (s,p,o) \in \Psi[r^* \leftarrow p^*](\sigma_1),\ g = URI(\Psi)\}$$

It then follows that

$$q \in S_1[\Psi](p^*) \wedge q \notin S_2[\Psi](p^*)$$

By first conjunct, q can be derived from p^* using Ψ in σ_0. By Lemma 1, the pivot tuple p^* and the rule Ψ are the unique way of deriving q in σ_0. Furthermore, the subject of q is $URI(p^*)$. But, since $p^* \in A^-[\Psi](u)$, this derivation is invalidated by the deletion of the tuples in D from $R(\sigma_0)$. Hence, there would be no pivot tuple $t \in R^*(\sigma_0)$ and no rule $\Phi \in M$ that derives q in σ_0, after the deletion of the tuples in D from $R(\sigma_0)$. In particular, there would be no pivot tuple $t \in R^*(\sigma_0)$ such that the subject of q is $URI(t)$, after the deletion of the tuples in D from $R(\sigma_0)$. Hence, if $q \in S_1[\Psi](p^*)$, then q has no derivation in σ_0, after the deletion of the tuples in D from $R(\sigma_0)$.

However, after inserting the tuples in I into $R(\sigma_0)$, it might be the case that q could be derived from the pivot tuple p^* and the rule Ψ in σ_1. But any such quad would be in $S_2[\Psi](p^*)$ and, therefore, removed from $S_1[\Psi](p^*)$.

Hence, if $q \in S_1[\Psi](p^*) \wedge q \notin S_2[\Psi](p^*)$, then q has no derivation in σ_0, after the deletion of the tuples in D from $R(\sigma_0)$ and the insertion of the tuples in I

into $R(\sigma_0)$. But $R(\sigma_1) = (R(\sigma_0) \setminus D) \cup I)$. Hence, if $q \in S_1[\Psi](p^*) \wedge q \notin S_2[\Psi](p^*)$, then q has no derivation in σ_1, that is, $q \notin W_1$.

Thus, in either case, if $q \in \Delta^-(u)$ and $q \notin \Delta^+(u)$, then $q \notin W_1$.

Lemma 3 (Characterization of the Insertion Component).
$\Delta^+(u)$ is the set of quads whose derivation becomes valid in σ_1 due to some tuple $i \in I$.

Proof Sketch. Follows from Definitions 6 and 7.

Theorem 1 (Correctness of the Incremental Changesets).

$$W_1 = \big(W_0 \setminus \Delta^-(u)\big) \cup \Delta^+(u)$$

Proof. We will prove the set equality by showing mutual inclusion.

($\subseteq$) Assume that $q \in W_1$.

Case 1: Assume that $q \in W_0$.

Since $q \in W_1$, by Lemma 2, either $q \notin \Delta^-(u)$ or $q \in \Delta^+(u)$.

Assume $q \in \Delta^+(u)$. Then, trivially

$$q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

Assume $q \notin \Delta^+(u)$. Then, $q \notin \Delta^-(u)$. But, by assumption, $q \in W_0$. Hence, it follows that

$$q \in (W_0 \setminus \Delta^-(u))$$

Therefore, it follows that

$$q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

Case 2: Assume that $q \notin W_0$.

Then, since $q \in W_1$ but $q \notin W_0$, q became derivable in σ_1 due to some tuple $i \in I$. By Lemma 3, $q \in \Delta^+(u)$.

Therefore, in both cases, it follows that

$$q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u).$$

($\supseteq$) Assume that $q \in (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$.

Case 1: Assume that $q \in \Delta^+(u)$.

Then, by Lemma 3, q is derivable under σ_1, and thus $q \in W_1$.

Case 2: Assume that $q \in (W_0 \setminus \Delta^-(u))$.

Then, $q \in W_0$ and $q \notin \Delta^-(u)$. By Lemma 2, $q \in W_1$.

Therefore, in both cases, it follows that $q \in W_1$.

□

Let $u = (D, I)$ be an update. Recall that σ_0 and σ_1 be the database states before and after applying u, and $W_1 = RDF\_State(\mathcal{W}, \sigma_1)$ and $W_0 = RDF\_State(\mathcal{W}, \sigma_0)$ are the view states before and after applying u. Let σ_D be the database state after applying the deletions in D, but before applying the insertions in I. Let $W_D = RDF\_State(\mathcal{W}, \sigma_D)$ be the view state corresponding to W_D.

Corollary 1 (Minimality of the Incremental Changesets).

(a) $\Delta^-(u) = (W_0 \setminus W_D)$
(b) $(\Delta^+(u) \setminus W_D) = (W_1 \setminus W_D)$
(c) If $(\Delta^+(u) \cap W_D) = \emptyset$, then $\Delta^+(u) = (W_1 \setminus W_D)$

Proof

First note that, executing $u = (D, I)$ on σ_0 is equivalent to executing the update $u_D = (D, \emptyset)$ on σ_0, creating σ_D, and then executing the update $u_I = (\emptyset, I)$ on σ_D, creating σ_1.

Part (a): By Theorem 1, since the insertion component of u_D is empty, we have

$$W_D = (W_0 \setminus \Delta^-(u))$$

Hence

$$(W_0 \setminus W_D) = (W_0 \setminus \ (W_0 \setminus \Delta^-(u)))$$

But $\Delta^-(u) \subseteq W_0$. Hence

$$(W_0 \setminus W_D) = \Delta^-(u)$$

Part (b): By Theorem 1, since the deletion component of u_I is empty, we have

$$W_1 = W_D \cup \Delta^+(u)$$

Hence

$$(W_1 \setminus W_D) = (W_D \cup \Delta^+(u)) \setminus W_D$$

Hence

$$(W_1 \setminus W_D) = (\Delta^+(u) \setminus W_D)$$

Part (c): First observe that, if $(\Delta^+(u) \cap W_D) = \emptyset$, then $(\Delta^+(u) \setminus W_D) = \Delta^+(u)$. Hence, from Part (b), it follows that

$$(W_1 \setminus W_D) = (\Delta^+(u) \setminus W_D) = \Delta^+(u)$$

□

References

1. Abo Khamis, M., Chmielewski, E., Draghici, A., Kara, A., Olteanu, D.: Maintaining queries under updates using heavy-light partitioning of the input relations. Proc. ACM Manag. Data **4**(2) (2026). https://doi.org/10.1145/3801905
2. Ali, M.A., Fernandes, A.A.A., Paton, N.W.: Movie: An incremental maintenance system for materialized object views. Data Knowl. Eng. **47**(2), 131–166 (Nov 2003)
3. Anonymized1: Anonymized reference 1.
4. Anonymized2: Anonymized reference 2.
5. Anonymized3: Anonymized reference 3.
6. Augusto, C., Bertolino, A., Angelis, G.D., Lonetti, F., Morán, J.: Large language models for software testing: A research roadmap (2025), https://arxiv.org/abs/2509.25043
7. Battiston, I., Kathuria, K., Boncz, P.: Openivm: a sql-to-sql compiler for incremental computations. In: Companion of the 2024 International Conference on Management of Data. p. 516–519. ACM (Jun 2024). https://doi.org/10.1145/3626246.3654743
8. Beg, A., O'Donoghue, D., Monahan, R.: Leveraging llms for formal software requirements – challenges and prospects (2025), https://arxiv.org/abs/2507.14330
9. Budiu, M., Chajed, T., McSherry, F., Ryzhyk, L., Tannen, V.: Dbsp: Automatic incremental view maintenance for rich query languages. Proc. VLDB Endow. **16**(7), 1601–1614 (Mar 2023). https://doi.org/10.14778/3587136.3587137
10. Ceri, S., Widom, J.: Deriving production rules for incremental view maintenance. In: Proceedings of the 17th International Conference on Very Large Data Bases. pp. 577–589. VLDB '91, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1991)
11. Councilman, A., Fu, D.J., Gupta, A., Wang, C., Grove, D., Wang, Y.X., Adve, V.: Towards formal verification of llm-generated code from natural language prompts (2025), https://arxiv.org/abs/2507.13290
12. Ferrari, A., Spoletini, P.: Formal requirements engineering and large language models: A two-way roadmap. Information and Software Technology **181**, 107697 (2025). https://doi.org/https://doi.org/10.1016/j.infsof.2025.107697
13. Han, S., Ives, Z.G.: Implementing views for property graphs. SIGMOD Rec. **54**(1), 59–68 (2025). https://doi.org/10.1145/3733620.3733633
14. Hu, X., Wang, Q.: Towards update-dependent analysis of query maintenance. Proc. ACM Manag. Data **3**(2) (2025). https://doi.org/10.1145/3725254
15. Jin, X., Liao, H.: An algorithm for incremental maintenance of materialized xpath view. In: Proceedings of the 11th International Conference on Web-age Information Management. pp. 513–524. WAIM'10, Springer-Verlag, Berlin, Heidelberg (2010)

16. Le-Cong, T., Le, B., Murray, T.: Can LLMs reason about program semantics? a comprehensive evaluation of LLMs on formal specification inference. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 21991–22014. Association for Computational Linguistics, Vienna, Austria (Jul 2025). https://doi.org/10.18653/v1/2025.acl-long.1068
17. Li, P., Glavic, B., Gawlick, D., Krishnaswamy, V., Liu, Z.H., Porobic, D., Niu, X.: In-memory incremental maintenance of provenance sketches. In: Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24-27, 2026. pp. 56–68. OpenProceedings.org (2026). https://doi.org/10.48786/EDBT.2026.05
18. Liu, X., Shen, S., Li, B., Ma, P., Jiang, R., Zhang, Y., Fan, J., Li, G., Tang, N., Luo, Y.: A survey of text-to-sql in the era of llms: Where are we, and where are we going? IEEE Transactions on Knowledge and Data Engineering **37**(10), 5735–5754 (2025). https://doi.org/10.1109/TKDE.2025.3592032
19. Lukyanenko, R., et al.: Manifesto for software development and modeling in the age of artificial intelligence. SSRN Electronic Journal (12 2025). https://doi.org/10.2139/ssrn.5881104
20. Luo, Y., Li, G., Fan, J., Chai, C., Tang, N.: Natural language to sql: State of the art and open problems. Proc. VLDB Endow. **18**(12), 5466–5471 (Aug 2025). https://doi.org/10.14778/3750601.3750696
21. Mitchell, J., Kim, B.H., Zhou, C., Wang, C.: Understanding formal reasoning failures in llms as abstract interpreters. In: Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages. p. 71–83. LMPL '25, Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3759425.3763389
22. Mitchell, J.L., Kim, B.H., Zhou, C., Wang, C.: Can llms formally reason as abstract interpreters for program analysis? CoRR **abs/2503.12686** (03 2025). https://doi.org/10.48550/arXiv.2503.12686
23. Motschnig-Pitrik, R.: The viewpoint abstraction in object-oriented modeling and the UML. In: Proceedings of the 19th International Conference on Conceptual Modeling. pp. 543–557. ER'00, Springer-Verlag, Berlin, Heidelberg (2000). https://doi.org/10.1007/3-540-45393-8_39
24. Murphy, W., Holzer, N., Qiao, F., Cui, L., Rothkopf, R., Koenig, N., Santolucito, M.: Combining llm code generation with formal specifications and reactive program synthesis (2024), https://arxiv.org/abs/2410.19736
25. Olteanu, D.: Recent increments in incremental view maintenance (2024), https://arxiv.org/abs/2404.17679
26. Poggi, A., Lembo, D., Calvanese, D., De Giacomo, G., Lenzerini, M., Rosati, R.: Linking data to ontologies. In: Spaccapietra, S. (ed.) Journal on Data Semantics X, pp. 133–173. Springer-Verlag, Berlin, Heidelberg (2008), http://dl.acm.org/citation.cfm?id=1793934.1793939
27. Sequeda, J.F., Arenas, M., Miranker, D.P.: OBDA: query rewriting or materialization? in practice, both! In: The Semantic Web - ISWC 2014 - 13th International Semantic Web Conference, October 19-23, 2014. Proceedings, Part I. pp. 535–551. Springer, Cham, Riva del Garda, Italy (2014). https://doi.org/10.1007/978-3-319-11964-9_34
28. Shi, L., Tang, Z., Zhang, N., Zhang, X., Yang, Z.: A survey on employing large language models for text-to-sql tasks. ACM Comput. Surv. (Jun 2025). https://doi.org/10.1145/3737873

29. Singh, A., Shetty, A., Ehtesham, A., Kumar, S., Khoei, T.T.: A survey of large language model-based generative AI for text-to-SQL: Benchmarks, applications, use cases, and challenges. In: 2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC). pp. 00015–00021 (2025). https://doi.org/10.1109/CCWC62904.2025.10903689
30. Singh, V., Cassel, D., Weir, N., Feng, N., Bayless, S.: Verge: Formal refinement and guidance engine for verifiable llm reasoning (2026), https://arxiv.org/abs/2601.20055
31. Staquet, D., Buelens, B., Van den Bussche, J.: Incremental view maintenance for sparql queries: Adapting the counting algorithm. ACM Trans. Web (Feb 2026). https://doi.org/10.1145/3796549
32. Svingos, C., Hernich, A., Gildhoff, H., Papakonstantinou, Y., Ioannidis, Y.: Foreign keys open the door for faster incremental view maintenance. Proc. ACM Manag. Data **1**(1) (2023). https://doi.org/10.1145/3588720
33. Swartz, A.: Musicbrainz: a semantic web service. IEEE Intelligent Systems **17**(1), 76–77 (2002). https://doi.org/10.1109/5254.988466
34. Vandenbrande, M., Taelman, R., Bonte, P., Ongenae, F.: Incremunica: Web-based incremental view maintenance for sparql. In: Curry, E., Acosta, M., Poveda-Villalón, M., van Erp, M., Ojo, A., Hose, K., Shimizu, C., Lisena, P. (eds.) The Semantic Web. pp. 297–315. Springer Nature Switzerland, Cham (2025)
35. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, agents, and vision. IEEE Trans. Softw. Eng. **50**(4), 911–936 (Apr 2024). https://doi.org/10.1109/TSE.2024.3368208
36. Wang, K., Mao, B., Jia, S., Ding, Y., Han, D., Ma, T., Cao, B.: Sgcr: A specification-grounded framework for trustworthy llm code review (2026), https://arxiv.org/abs/2512.17540
37. Yadav, R., Abeysinghe, S., Yang, M., Helt, J., Ung, M., Chen, Y., Hu, M., Wei, W., Yang, Y., van Bussel, T., Chatterji, S., Roy, I., Lappas, P., Papakonstantinou, Y., Fayyaz, T., Aslam, B., Bunker, R., Armbrust, M., Shankar, S.: Enzyme: Incremental view maintenance for data engineering (2026), https://arxiv.org/abs/2603.27775
38. Zhang, Y., et al.: Position: Trustworthy AI agents require the integration of large language models and formal methods. In: Proceedings of the 42nd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 267, pp. 82441–82459. PMLR (13–19 Jul 2025), https://proceedings.mlr.press/v267/zhang25ds.html
39. Zhao, W., Rusu, F., Dong, B., Wu, K., Nugent, P.: Incremental view maintenance over array data. In: Proceedings of the 2017 ACM International Conference on Management of Data. pp. 139–154. SIGMOD '17, ACM, New York, NY, USA (2017)